

Source Code

Frontend\src\App.jsx

```
import React from "react";
import Navbar from "../components/pages/Navbar";
import Login from "../components/authentication/Login";
import Register from "../components/authentication/Register";
import { createBrowserRouter, RouterProvider } from "react-router-dom";
import Home from "../components/pages/Home";
import PrivacyPolicy from "../components/pages/PrivacyPolicy.jsx";
import TermsOfService from "../components/pages/TermsOfService.jsx";
import InternshipListings from "../components/pages/InternshipListings.jsx";
import Browse from "../components/pages/Browse.jsx";
import Profile from "../components/pages/Profile.jsx";
import InternshipDetails from "../components/pages/InternshipDetails.jsx";
import Companies from "../components/admin_dashboard/Companies";
import CompanyCreate from "../components/admin_dashboard/CompanyCreate";
import CompanySetup from "../components/admin_dashboard/CompanySetup";
import AdminJobs from "../components/admin_dashboard/AdminJobs.jsx";
import PostJob from "../components/admin_dashboard/PostJob";
import Applicants from "../components/admin_dashboard/Applicants";
import ProtectedRoute from "../components/admin_dashboard/ProtectedRoute";
import Creator from "../components/creator/Creator.jsx";

const appRouter = createBrowserRouter([
  { path: "/", element: <Home /> },
  {
    path: "/login",
    element: <Login />,
  },
  {
    path: "/register",
    element: <Register />,
  },
  {
    path: "/description/:id",
    element: <InternshipDetails />,
  },
  {
    path: "/Profile",
    element: <Profile />,
  },
  {
    path: "/PrivacyPolicy",
    element: <PrivacyPolicy />,
  },
  {
    path: "/TermsOfService",
    element: <TermsOfService />,
  },
  {
    path: "/Jobs",
    element: <InternshipListings />,
  },
  {
    path: "/Home",
    element: <Home />,
  },
  {
    path: "/Browse",
    element: <Browse />,
  },
  {
    path: "/Creator",
    element: <Creator />,
  },
  // /admin
  {
    path: "/admin/companies",
    element: (
      <ProtectedRoute>
        <Companies />
      </ProtectedRoute>
    ),
  },
  {
    path: "/admin/companies/create",
    element: (
      <ProtectedRoute>
        <CompanyCreate />
      </ProtectedRoute>
    ),
  },
]);
```

```

    },
    {
      path: "/admin/companies/:id",
      element: (
        <ProtectedRoute>
          <CompanySetup />
        </ProtectedRoute>
      ),
    },
    {
      path: "/admin/jobs",
      element: (
        <ProtectedRoute>
          { " "}
          <AdminJobs />
        </ProtectedRoute>
      ),
    },
    {
      path: "/admin/jobs/create",
      element: (
        <ProtectedRoute>
          { " "}
          <PostJob />{ " "}
        </ProtectedRoute>
      ),
    },
    {
      path: "/admin/jobs/:id/applicants",
      element: (
        <ProtectedRoute>
          <Applicants />
        </ProtectedRoute>
      ),
    },
  ],
]);

```

```

function App() {
  return (
    <div>
      <RouterProvider router={appRouter}></RouterProvider>
    </div>
  );
}

```

```

export default App;

```

Frontend\src\components\admin_dashboard\AdminJobs.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../pages/Navbar";
import { Input } from "../ui/input";
import { Button } from "../ui/button";
import { useNavigate } from "react-router-dom";
import { useDispatch } from "react-redux";
import AdminJobsTable from "../AdminJobsTable";
import useGetAllAdminJobs from "@/hooks/useGetAllJAdminobs";
import { setSearchJobByText } from "@redux/jobSlice";

const AdminJobs = () => {
  useGetAllAdminJobs();
  const navigate = useNavigate();

  const [input, setInput] = useState("");
  const dispatch = useDispatch();

  useEffect(() => {
    dispatch(setSearchJobByText(input));
  }, [input]);
  return (
    <div>
      <Navbar />
      <div className="max-w-6xl mx-auto my-10">
        <div className="flex items-center justify-between my-5">
          <Input
            className="w-fit"
            placeholder="Filter by Name & Jobs"
            onChange={(e) => setInput(e.target.value)}
          />
          <Button onClick={() => navigate("/admin/jobs/create")}>
            Post new Job
          </Button>
        </div>
        <div>
          <AdminJobsTable />
        </div>
      </div>
    </div>
  );
};

export default AdminJobs;

```

Frontend\src\components\admin_dashboard\AdminJobsTable.jsx

```

import React, { useEffect, useState } from "react";
import {
  Table,
  TableBody,
  TableCaption,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "../ui/table";

import { Popover, PopoverContent, PopoverTrigger } from "../ui/popover";
import { Edit2, Eye, MoreHorizontal } from "lucide-react";
import { useSelector } from "react-redux";
import { useNavigate } from "react-router-dom";

const AdminJobsTable = () => {
  const { companies, searchCompanyByText } = useSelector(
    (store) => store.company
  );
  const { allAdminJobs, searchJobByText } = useSelector((store) => store.job);
  const navigate = useNavigate();

  const [filterJobs, setFilterJobs] = useState(allAdminJobs);

  useEffect(() => {
    const filteredJobs =
      allAdminJobs.length >= 0 &&
      allAdminJobs.filter((job) => {
        if (!searchJobByText) {
          return true;
        }
        return (
          job.title?.toLowerCase().includes(searchJobByText.toLowerCase()) ||
          job?.company?.name
            .toLowerCase()
            .includes(searchJobByText.toLowerCase())
        );
      });
  });
};

```



```

import React, { useEffect } from "react";
import ApplicantsTable from "../ApplicantsTable";
import axios from "axios";
import { useParams } from "react-router-dom";
import { useDispatch, useSelector } from "react-redux";
import { setAllApplicants } from "@redux/applicationSlice";
import { APPLICATION_API_ENDPOINT } from "@utils/data";
import Navbar from "../pages/Navbar";

const Applicants = () => {
  const params = useParams();
  const dispatch = useDispatch();
  const { applicants } = useSelector((store) => store.application);

  useEffect(() => {
    const fetchAllApplicants = async () => {
      try {
        const res = await axios.get(
          `${APPLICATION_API_ENDPOINT}/${params.id}/applicants`,
          { withCredentials: true }
        );
        dispatch(setAllApplicants(res.data.job));
        console.log(res.data);
      } catch (error) {
        console.log(error);
      }
    };
    fetchAllApplicants();
  }, []);
  return (
    <div>
      <Navbar />
      <div className="max-w-7xl mx-auto">
        <h1 className="font-bold text-xl my-5">
          Applicants {applicants?.applications?.length}
        </h1>
        <ApplicantsTable />
      </div>
    </div>
  );
};

export default Applicants;

```

Frontend\src\components\admin_dashboard\ApplicantsTable.jsx

```

import React from "react";
import {
  Table,
  TableBody,
  TableCaption,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "../ui/table";
import { Popover, PopoverContent, PopoverTrigger } from "../ui/popover";
import { MoreHorizontal } from "lucide-react";
import { useSelector } from "react-redux";
import { toast } from "sonner";
import axios from "axios";
import { APPLICATION_API_ENDPOINT } from "@utils/data";

const shortlistingStatus = ["Accepted", "Rejected"];

const ApplicantsTable = () => {
  const { applicants } = useSelector((store) => store.application);

  const statusHandler = async (status, id) => {
    console.log("called");
    try {
      axios.defaults.withCredentials = true;
      const res = await axios.post(
        `${APPLICATION_API_ENDPOINT}/${status}/${id}/update`,
        { status }
      );
      console.log(res);
      if (res.data.success) {
        toast.success(res.data.message);
      }
    } catch (error) {
      toast.error(error.response.data.message);
    }
  };
};

```

```

return (
  <div>
    <Table>
      <TableCaption>A list of your recent applied user</TableCaption>
      <TableHeader>
        <TableRow>
          <TableHead>FullName</TableHead>
          <TableHead>Email</TableHead>
          <TableHead>Contact</TableHead>
          <TableHead>Resume</TableHead>
          <TableHead>Date</TableHead>
          <TableHead className="text-right">Action</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {applicants &&
          applicants?.applications?.map((item) => (
            <tr key={item._id}>
              <TableCell>{item?.applicant?.fullname}</TableCell>
              <TableCell>{item?.applicant?.email}</TableCell>
              <TableCell>{item?.applicant?.phoneNumber}</TableCell>
              <TableCell>
                {item?.applicant?.profile?.resume ? (
                  <a
                    className="text-blue-600 cursor-pointer"
                    href={item?.applicant?.profile?.resume}
                    target="_blank"
                    rel="noopener noreferrer"
                  >
                    Download
                    {`/* {item?.applicant?.profile?.resume} */`}
                  </a>
                ) : (
                  <span>NA</span>
                )}
              </TableCell>
              <TableCell>{item?.applicant?.createdAt.split("T")[0]}</TableCell>
              <TableCell className="float-right cursor-pointer">
                <Popover>
                  <PopoverTrigger>
                    <MoreHorizontal />
                  </PopoverTrigger>
                  <PopoverContent className="w-32">
                    {shortlistingStatus.map((status, index) => {
                      return (
                        <div>
                          onClick={() => statusHandler(status, item?._id)}
                          key={index}
                          className="flex w-fit items-center my-2 cursor-pointer"
                        >
                          <input
                            type="radio"
                            name="shortlistingStatus"
                            value={status}
                          />{` `}
                          {status}
                        </div>
                      );
                    })}
                  </PopoverContent>
                </Popover>
              </TableCell>
            </tr>
          ))}
        </TableBody>
      </Table>
    </div>
  );
};

export default ApplicantsTable;

```

Frontend\src\components\admin_dashboard\Companies.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../pages/Navbar";
import { Input } from "../ui/input";
import { Button } from "../ui/button";
import CompaniesTable from "../CompaniesTable";
import { useNavigate } from "react-router-dom";

import useGetAllCompanies from "@/hooks/useGetAllCompanies";
import { useDispatch } from "react-redux";
import { setSearchCompanyByText } from "@redux/companyslice";

const Companies = () => {
  const navigate = useNavigate();

  useGetAllCompanies();
  const [input, setInput] = useState("");
  const dispatch = useDispatch();

  useEffect(() => {
    dispatch(setSearchCompanyByText(input));
  }, [input]);
  return (
    <div>
      <Navbar />
      <div className=" max-w-6xl mx-auto my-10">
        <div className="flex items-center justify-between my-5">
          <Input
            className="w-fit"
            placeholder="Filter by Name"
            onChange={(e) => setInput(e.target.value)}
          ></Input>
          <Button onClick={() => navigate("/admin/companies/create")}>
            Add Company
          </Button>
        </div>
        <div>
          <CompaniesTable />
        </div>
      </div>
    </div>
  );
};

export default Companies;

```

Frontend\src\components\admin_dashboard\CompaniesTable.jsx

```

import React, { useEffect, useState } from "react";
import {
  Table,
  TableBody,
  TableCaption,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "../ui/table";
import { Avatar, AvatarImage } from "../ui/avatar";
import { Popover, PopoverContent, PopoverTrigger } from "../ui/popover";
import { Edit2, MoreHorizontal } from "lucide-react";
import { useSelector } from "react-redux";
import { useNavigate } from "react-router-dom";

const CompaniesTable = () => {
  const { companies, searchCompanyByText } = useSelector(
    (store) => store.company
  );
  const navigate = useNavigate();
  const [filterCompany, setFilterCompany] = useState(companies);

  useEffect(() => {
    const filteredCompany =
      companies.length >= 0 &&
      companies.filter((company) => {
        if (!searchCompanyByText) {
          return true;
        }
        return company.name
          ?.toLowerCase()
          .includes(searchCompanyByText.toLowerCase());
      });
    setFilterCompany(filteredCompany);
  }, [companies, searchCompanyByText]);

  console.log("COMPANIES", companies);

```

```

if (!companies) {
  return <div>Loading...</div>;
}

return (
  <div>
    <Table>
      <TableCaption>Your recent registered Companies</TableCaption>
      <TableHeader>
        <TableRow>
          <TableHead>Logo</TableHead>
          <TableHead>Company Name</TableHead>
          <TableHead>Date</TableHead>
          <TableHead className="text-right">Action</TableHead>
        </TableRow>
      </TableHeader>

      <TableBody>
        {filterCompany.length === 0 ? (
          <span>No Companies Added</span>
        ) : (
          filterCompany?.map((company) => (
            <TableRow key={company.id}>
              <TableCell>
                <Avatar>
                  <AvatarImage
                    src={company.logo || "default-logo-url"}
                    alt={` ${company.name} logo`}
                  />
                </Avatar>
              </TableCell>
              <TableCell>{company.name}</TableCell>
              <TableCell>{company.createdAt.split("T")[0]}</TableCell>
              <TableCell className="text-right cursor-pointer">
                <Popover>
                  <PopoverTrigger>
                    <MoreHorizontal />
                  </PopoverTrigger>
                  <PopoverContent className="w-32">
                    <div>
                      onClick={() => navigate(`/admin/companies/${company._id}`)}
                      className="flex items-center gap-2 w-fit cursor-pointer"
                    >
                      <Edit2 className="w-4" />
                      <span>Edit</span>
                    </div>
                  </PopoverContent>
                </Popover>
              </TableCell>
            </TableRow>
          ))
        )}
      </TableBody>
    </Table>
  </div>
);
};

export default CompaniesTable;

```

Frontend\src\components\admin_dashboard\CompanyCreate.jsx


```

import React, { useState } from "react";
import Navbar from "../pages/Navbar";
import { Label } from "../ui/label";
import { Input } from "../ui/input";
import { Button } from "../ui/button";
import { useNavigate } from "react-router-dom";
import { COMPANY_API_ENDPOINT } from "@/utils/data";
import { toast } from "sonner";
import { useDispatch } from "react-redux";
import { setSingleCompany } from "@redux/companyslice";
import axios from "axios";

const CompanyCreate = () => {
  const navigate = useNavigate();
  const [companyName, setCompanyName] = useState();
  const dispatch = useDispatch();
  const registerNewCompany = async () => {
    try {
      const res = await axios.post(
        `${COMPANY_API_ENDPOINT}/register`,
        { companyName },
        {
          headers: {
            "Content-Type": "application/json",
          },
          withCredentials: true,
        }
      );
      if (res?.data?.success) {
        dispatch(setSingleCompany(res.data.company));
        toast.success(res.data.message);
        const companyId = res?.data?.company?._id;
        navigate(`/admin/companies/${companyId}`);
      }
    } catch (error) {
      console.log(error);
    }
  };
  return (
    <div>
      <Navbar />
      <div className="max-w-4xl mx-auto">
        <div className="my-10">
          <h1 className="font-bold text-2xl">Company Name</h1>
          <p className="text-gray-600">Company Description</p>
        </div>
        <Label>Company Name</Label>
        <Input
          type="text"
          placeholder="Company Name"
          className="my-2"
          onChange={ (e) => setCompanyName(e.target.value)}
        />

        <div className="flex items-center gap-2 my-10">
          <Button
            variant="outline"
            onClick={() => navigate("/admin/companies")}
          >
            Cancel
          </Button>
          <Button onClick={registerNewCompany}>Continue</Button>
        </div>
      </div>
    </div>
  );
};

export default CompanyCreate;

```

Frontend\src\components\admin_dashboard\CompanySetup.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../pages/Navbar.jsx";
import { Button } from "../ui/button.jsx";
import { ArrowLeft, Loader2 } from "lucide-react";
import { Label } from "../ui/label.jsx";
import { Input } from "../ui/input.jsx";
import axios from "axios";
import { COMPANY_API_ENDPOINT } from "../../utils/data.js";
import { useNavigate, useParams } from "react-router-dom";
import { toast } from "sonner";
import { useSelector } from "react-redux";
import useGetCompanyById from "@/hooks/useGetCompanyById.jsx";

```

```

const CompanySetup = () => {
  const params = useParams();
  useGetCompanyById(params.id);
  const [input, setInput] = useState({
    name: "",
    description: "",
    website: "",
    location: "",
    file: null,
  });
  const { singleCompany } = useSelector((store) => store.company);

  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const changeEventHandler = (e) => {
    setInput({ ...input, [e.target.name]: e.target.value });
  };

  const changeFileHandler = (e) => {
    const file = e.target.files?.[0];
    setInput({ ...input, file });
  };

  const submitHandler = async (e) => {
    e.preventDefault();
    const formData = new FormData();
    formData.append("name", input.name);
    formData.append("description", input.description);
    formData.append("website", input.website);
    formData.append("location", input.location);
    if (input.file) {
      formData.append("file", input.file);
    }
    try {
      setLoading(true);
      const res = await axios.put(
        `${COMPANY_API_ENDPOINT}/update/${params.id}`,
        formData,
        {
          headers: {
            "Content-Type": "multipart/form-data",
          },
          withCredentials: true,
        }
      );
      console.log(res); // Debugging API response

      // Assuming a successful response has a `message` property
      if (res.status === 200 && res.data.message) {
        toast.success(res.data.message);
        navigate("/admin/companies");
      } else {
        throw new Error("Unexpected API response.");
      }
    } catch (error) {
      const errorMessage =
        error.response?.data?.message || "An unexpected error occurred.";
      toast.error(errorMessage);
    } finally {
      setLoading(false);
    }
  };
};

useEffect(() => {
  setInput({
    name: singleCompany.name || "",
    description: singleCompany.description || "",
    website: singleCompany.website || "",
    location: singleCompany.location || "",
    file: singleCompany.file || null,
  });
}, [singleCompany]);

return (
  <div>
    <Navbar />
    <div className="max-w-xl mx-auto my-10">
      <form onSubmit={submitHandler}>
        <div className="flex items-center gap-5 p-8">
          <Button
            onClick={() => navigate("/admin/companies")}
            variant="outline"
            className="flex items-center gap-2 text-gray-500 font-semibold"
          >
            <ArrowLeft />

```

```

        <span>Back</span>
      </Button>
    <h1 className="font-bold text-xl">Company Setup</h1>
  </div>
  <div className="grid grid-cols-2 gap-4">
    <div>
      <Label>Company Name</Label>
      <Input
        type="text"
        name="name"
        value={input.name}
        onChange={changeEventHandler}
      />
    </div>
    <div>
      <Label>Description</Label>
      <Input
        type="text"
        name="description"
        value={input.description}
        onChange={changeEventHandler}
      />
    </div>
    <div>
      <Label>Website</Label>
      <Input
        type="text"
        name="website"
        value={input.website}
        onChange={changeEventHandler}
      />
    </div>
    <div>
      <Label>Location</Label>
      <Input
        type="text"
        name="location"
        value={input.location}
        onChange={changeEventHandler}
      />
    </div>
    <div>
      <Label>Logo</Label>
      <Input
        type="file"
        accept="image/*"
        onChange={changeFileHandler}
      />
    </div>
  </div>
  {loading ? (
    <Button className="w-full my-4">
      { " " }
    <Loader2 className="mr-2 h-4 w-4 animate-spin" /> Please wait{ " " }
    </Button>
  ) : (
    <Button type="submit" className="w-full my-4">
      Update
    </Button>
  )}
</form>
</div>
</div>
);
};

export default CompanySetup;

```

Frontend\src\components\admin_dashboard\PostJob.jsx

```

import React, { useState } from "react";
import Navbar from "../pages/Navbar";
import { Input } from "../ui/input";
import { Label } from "../ui/label";
import { Button } from "../ui/button";
import { useSelector } from "react-redux";
import store from "@redux/store";
import {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectTrigger,
  SelectValue,

```

```

} from "../ui/select";
import axios from "axios";
import { JOB_API_ENDPOINT } from "@/utils/data";
import { toast } from "sonner";
import { useNavigate } from "react-router-dom";
import { Loader2 } from "lucide-react";

const companyArray = [];

const PostJob = () => {
  const [input, setInput] = useState({
    title: "",
    description: "",
    requirements: "",
    salary: "",
    location: "",
    jobType: "",
    experience: "",
    position: 0,
    companyId: "",
  });
  const navigate = useNavigate();
  const { companies } = useSelector((store) => store.company);
  const changeEventHandler = (e) => {
    setInput({ ...input, [e.target.name]: e.target.value });
  };
  const [loading, setLoading] = useState(false);

  const selectChangeHandler = (value) => {
    const selectedCompany = companies.find(
      (company) => company.name.toLowerCase() === value
    );
    setInput({ ...input, companyId: selectedCompany._id });
  };

  const submitHandler = async (e) => {
    e.preventDefault();
    try {
      setLoading(true);
      const res = await axios.post(`${JOB_API_ENDPOINT}/post`, input, {
        headers: {
          "Content-Type": "application/json",
        },
        withCredentials: true,
      });
      if (res.data.success) {
        toast.success(res.data.message);
        navigate("/admin/jobs");
      } else {
        toast.error(res.data.message);
        navigate("/admin/jobs");
      }
    } catch (error) {
      if (error.response && error.response.data) {
        toast.error(error.response.data.message || "Something went wrong");
      } else {
        toast.error("An unexpected error occurred");
      }
    }
  } finally {
    setLoading(false);
  }
};

return (
  <div>
    <Navbar />
    <div className="flex items-center justify-center w-screen my-5">
      <form
        onSubmit={submitHandler}
        className="p-8 max-w-4xl border border-gray-500 shadow-sm hover:shadow-xl hover:shadow-red-300 rounded-lg"
      >
        <div className="grid grid-cols-2 gap-5">
          <div>
            <Label>Title</Label>
            <Input
              type="text"
              name="title"
              value={input.title}
              placeholder="Enter job title"
              className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
              onChange={changeEventHandler}
            />
          </div>
          <div>
            <Label>Description</Label>

```

```

<Input
  name="description"
  value={input.description}
  placeholder="Enter job description"
  className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400 "
  onChange={changeEventHandler}
/>
</div>
<div>
  <Label>Location</Label>
  <Input
    type="text"
    name="location"
    value={input.location}
    placeholder="Enter job location"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>
<div>
  <Label>Salary</Label>
  <Input
    type="number"
    name="salary"
    value={input.salary}
    placeholder="Enter job salary"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>
<div>
  <Label>Position</Label>
  <Input
    type="number"
    name="position"
    value={input.position}
    placeholder="Enter job position"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>
<div>
  <Label>Requirements</Label>
  <Input
    type="text"
    name="requirements"
    value={input.requirements}
    placeholder="Enter job requirements"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>

<div>
  <Label>Experience</Label>
  <Input
    type="number"
    name="experience"
    value={input.experience}
    placeholder="Enter job experience"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>
<div>
  <Label>Job Type</Label>
  <Input
    type="text"
    name="jobType"
    value={input.jobType}
    placeholder="Enter job type"
    className="focus-visible:ring-offset-0 focus-visible:ring-0 my-1 hover:shadow-blue-400"
    onChange={changeEventHandler}
  />
</div>

<div>
  {companies.length > 0 && (
    <Select onChange={selectChangeHandler}>
      <SelectTrigger className="w-[180px]">
        <SelectValue placeholder="Select a Company" />
      </SelectTrigger>
      <SelectContent>
        <SelectGroup>
          {companies.map((company) => (
            <SelectItem

```

```

                key={company._id}
                value={company.name.toLowerCase()}
            >
                {company.name}
            </SelectedItem>
        )))
    </SelectGroup>
</SelectContent>
</Select>
    )}
</div>
</div>
<div className="flex items-center justify-center mt-5">
    {loading ? (
        <Button className="w-full px-4 py-2 text-sm text-white bg-black rounded-md ">
            { " " }
            <Loader2 className="mr-2 h-4 w-4 animate-spin" /> Please wait{ " " }
        </Button>
    ) : (
        <Button
            type="submit"
            className="w-full px-4 py-2 text-sm text-white bg-black rounded-md hover:bg-blue-600"
        >
            Post Job
        </Button>
    )}
</div>
{companies.length === 0 && (
    <p className="text-sm font-bold my-3 text-center text-red-600">
        *Please register a company to post jobs.*
    </p>
)}
</form>
</div>
</div>
);
};

export default PostJob;

```

Frontend\src\components\admin_dashboard\ProtectedRoute.jsx

```

import { useEffect } from "react";
import { useSelector } from "react-redux";
import { useNavigate } from "react-router-dom";

const ProtectedRoute = ({ children }) => {
    const { user } = useSelector((store) => store.auth);
    const navigate = useNavigate();

    useEffect(() => {
        if (!user || user.role !== "Recruiter") {
            navigate("/");
        }
    }, [user, navigate]);

    if (!user || user.role !== "Recruiter") {
        return null;
    }

    return <>{children}</>;
};

export default ProtectedRoute;

```

Frontend\src\components\authentication\Login.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../pages/Navbar";
import { Label } from "../ui/label";
import { Input } from "../ui/input";
import { Navigate, useNavigate } from "react-router-dom";
import { RadioGroup } from "../ui/radio-group";
import { Link } from "react-router-dom";
import axios from "axios";
import { toast } from "sonner";
import { USER_API_ENDPOINT } from "@/utils/data.js";
import { useDispatch, useSelector } from "react-redux";
import { setLoading, setUser } from "@redux/authSlice";

const Login = () => {
    const [input, setInput] = useState({

```

```

    email: "",
    password: "",
    role: "Student",
  });
  const navigate = useNavigate();
  const dispatch = useDispatch();
  const { loading, user } = useSelector((store) => store.auth);
  const changeEventHandler = (e) => {
    setInput({ ...input, [e.target.name]: e.target.value });
  };
  const ChangeFilehandler = (e) => {
    setInput({ ...input, file: e.target.files?.[0] });
  };

  const submitHandler = async (e) => {
    e.preventDefault();
    if (!input.email || !input.password) {
      toast.error("Please fill in both email and password.");
      return;
    }

    try {
      dispatch(setLoading(true)); // Start loading
      const res = await axios.post(`${USER_API_ENDPOINT}/login`, input, {
        headers: { "Content-Type": "application/json" },
        withCredentials: true,
      });
      if (res.data.success) {
        dispatch(setUser(res.data.user));
        navigate("/");
        toast.success(res.data.message);
      }
    } catch (error) {
      toast.error(error.response?.data?.message || "Login failed");
    } finally {
      dispatch(setLoading(false)); // End loading
    }
  };

  useEffect(() => {
    if (user) {
      navigate("/");
    }
  }, []);

  return (
    <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
      <Navbar />
      <div className="flex items-center justify-center pt-24 pb-10 px-4">
        <div className="w-full max-w-md animate-fade-in-up">
          <form
            onSubmit={submitHandler}
            className="glass-card p-8"
          >
            {/* Header */}
            <div className="text-center mb-8">
              <h1 className="text-3xl font-bold text-white mb-2">Welcome Back</h1>
              <p className="text-[#b3b3b3]">Sign in to your account</p>
            </div>

            {/* Email */}
            <div className="mb-4">
              <Label className="text-[#b3b3b3] text-sm mb-1 block">Email</Label>
              <Input
                type="email"
                value={input.email}
                name="email"
                onChange={changeEventHandler}
                placeholder="john.doe@gmail.com"
                className="dark-input w-full py-3 px-4"
              />
            </div>

            {/* Password */}
            <div className="mb-4">
              <Label className="text-[#b3b3b3] text-sm mb-1 block">Password</Label>
              <Input
                type="password"
                value={input.password}
                name="password"
                onChange={changeEventHandler}
                placeholder="....."
                className="dark-input w-full py-3 px-4"
              />
            </div>
          </form>
        </div>
      </div>
    </div>
  );

```

```

    /* Role selection */
    <div className="mb-6">
      <Label className="text-[#b3b3b3] text-sm mb-3 block">I am a</Label>
      <RadioGroup className="flex items-center gap-6">
        <label className="flex items-center gap-2 cursor-pointer group">
          <input
            type="radio"
            name="role"
            value="Student"
            checked={input.role === "Student"}
            onChange={changeEventHandler}
            className="accent-[#8b5cf6] w-4 h-4 cursor-pointer"
          />
          <span className="text-[#b3b3b3] group-hover:text-white transition-colors">Student</span>
        </label>
        <label className="flex items-center gap-2 cursor-pointer group">
          <input
            type="radio"
            name="role"
            value="Recruiter"
            checked={input.role === "Recruiter"}
            onChange={changeEventHandler}
            className="accent-[#8b5cf6] w-4 h-4 cursor-pointer"
          />
          <span className="text-[#b3b3b3] group-hover:text-white transition-colors">Recruiter</span>
        </label>
      </RadioGroup>
    </div>

    /* Submit */
    {loading ? (
      <div className="flex items-center justify-center py-4">
        <div className="w-8 h-8 border-2 border-[#8b5cf6] border-t-transparent rounded-full animate-spin"></div>
      </div>
    ) : (
      <button
        type="submit"
        className="portal-btn w-full py-3 text-base rounded-lg"
      >
        Sign In
      </button>
    )}

    /* Register link */
    <div className="text-center mt-6">
      <p className="text-[#b3b3b3]">
        New to Internship Portal?{" "}
        <Link to="/register" className="text-white font-semibold hover:text-[#8b5cf6] transition-colors">
          Sign up now
        </Link>
      </p>
    </div>
  </form>
</div>
</div>
);
};

export default Login;

```

Frontend\src\components\authentication\Register.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../pages/Navbar";
import { Label } from "../ui/label";
import { Input } from "../ui/input";
import { RadioGroup } from "../ui/radio-group";
import { Link, useNavigate } from "react-router-dom";
import axios from "axios";
import { USER_API_ENDPOINT } from "@/utils/data";
import { toast } from "sonner";
import { useDispatch, useSelector } from "react-redux";
import { setLoading } from "@redux/authSlice";

const Register = () => {
  const [input, setInput] = useState({
    fullname: "",
    email: "",
    password: "",
    role: "Student",
    phoneNumber: "",
    pancard: "",
    adharcard: "",
    file: "",
  });

```



```

));

const navigate = useNavigate();
const dispatch = useDispatch();
const { loading } = useSelector((store) => store.auth);

const changeEventHandler = (e) => {
  setInput({ ...input, [e.target.name]: e.target.value });
};
const ChangeFilehandler = (e) => {
  setInput({ ...input, file: e.target.files?.[0] });
};

const submitHandler = async (e) => {
  e.preventDefault();
  if (!input.fullname || !input.email || !input.password || !input.pancard || !input.adharcard || !input.phoneNumber) {
    toast.error("Please fill in all required fields.");
    return;
  }
  const formData = new FormData();
  formData.append("fullname", input.fullname);
  formData.append("email", input.email);
  formData.append("password", input.password);
  formData.append("pancard", input.pancard);
  formData.append("adharcard", input.adharcard);
  formData.append("role", input.role);
  formData.append("phoneNumber", input.phoneNumber);
  if (input.file) {
    formData.append("file", input.file);
  }
  try {
    dispatch(setLoading(true));
    const res = await axios.post(`${USER_API_ENDPOINT}/register`, formData, {
      headers: { "Content-Type": "multipart/form-data" },
      withCredentials: true,
    });
    if (res.data.success) {
      navigate("/login");
      toast.success(res.data.message);
    }
  } catch (error) {
    console.log(error);
    const errorMessage = error.response
      ? error.response.data.message
      : "An unexpected error occurred.";
    toast.error(errorMessage);
  } finally {
    dispatch(setLoading(false));
  }
};

const { user } = useSelector((store) => store.auth);
useEffect(() => {
  if (user) {
    navigate("/");
  }
}, []);

return (
  <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
    <Navbar />
    <div className="flex items-center justify-center pt-24 pb-10 px-4">
      <div className="w-full max-w-lg animate-fade-in-up">
        <form
          onSubmit={submitHandler}
          className="glass-card p-8"
        >
          {/* Header */}
          <div className="text-center mb-8">
            <h1 className="text-3xl font-bold text-white mb-2">Create Account</h1>
            <p className="text-[#b3b3b3]">Join the Internship Portal</p>
          </div>

          {/* Two column layout for name and email */}
          <div className="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
            <div>
              <Label className="text-[#b3b3b3] text-sm mb-1 block">Full Name</Label>
              <Input
                type="text"
                value={input.fullname}
                name="fullname"
                onChange={changeEventHandler}
                placeholder="John Doe"
                className="dark-input w-full py-3 px-4"
              />
            </div>

```

```

<div>
  <Label className="text-[#b3b3b3] text-sm mb-1 block">Email</Label>
  <Input
    type="email"
    value={input.email}
    name="email"
    onChange={changeEventHandler}
    placeholder="john.doe@gmail.com"
    className="dark-input w-full py-3 px-4"
  />
</div>
</div>

{/* Password */}
<div className="mb-4">
  <Label className="text-[#b3b3b3] text-sm mb-1 block">Password</Label>
  <Input
    type="password"
    value={input.password}
    name="password"
    onChange={changeEventHandler}
    placeholder="••••••••"
    className="dark-input w-full py-3 px-4"
  />
</div>

{/* PAN and Aadhar */}
<div className="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
  <div>
    <Label className="text-[#b3b3b3] text-sm mb-1 block">PAN Card Number</Label>
    <Input
      type="text"
      value={input.pancard}
      name="pancard"
      onChange={changeEventHandler}
      placeholder="ABCDEF1234G"
      className="dark-input w-full py-3 px-4"
    />
  </div>
  <div>
    <Label className="text-[#b3b3b3] text-sm mb-1 block">Aadhar Card Number</Label>
    <Input
      type="text"
      value={input.adharcard}
      name="adharcard"
      onChange={changeEventHandler}
      placeholder="123456789012"
      className="dark-input w-full py-3 px-4"
    />
  </div>
</div>

{/* Phone */}
<div className="mb-4">
  <Label className="text-[#b3b3b3] text-sm mb-1 block">Phone Number</Label>
  <Input
    type="tel"
    value={input.phoneNumber}
    name="phoneNumber"
    onChange={changeEventHandler}
    placeholder="+1234567890"
    className="dark-input w-full py-3 px-4"
  />
</div>

{/* Role */}
<div className="mb-4">
  <Label className="text-[#b3b3b3] text-sm mb-3 block">I am a</Label>
  <RadioGroup className="flex items-center gap-6">
    <label className="flex items-center gap-2 cursor-pointer group">
      <input
        type="radio"
        name="role"
        value="Student"
        checked={input.role === "Student"}
        onChange={changeEventHandler}
        className="accent-[#8b5cf6] w-4 h-4 cursor-pointer"
      />
      <span className="text-[#b3b3b3] group-hover:text-white transition-colors">Student</span>
    </label>
    <label className="flex items-center gap-2 cursor-pointer group">
      <input
        type="radio"
        name="role"
        value="Recruiter"
        checked={input.role === "Recruiter"}

```

```

        onChange={changeEventHandler}
        className="accent-[#8b5cf6] w-4 h-4 cursor-pointer"
      />
      <span className="text-[#b3b3b3] group-hover:text-white transition-colors">Recruiter</span>
    </label>
  </RadioGroup>
</div>

{/* Profile photo */}
<div className="mb-6">
  <Label className="text-[#b3b3b3] text-sm mb-1 block">Profile Photo</Label>
  <div className="dark-input py-2 px-4 rounded-lg">
    <Input
      type="file"
      accept="image/*"
      onChange={ChangeFilehandler}
      className="cursor-pointer border-0 bg-transparent text-[#b3b3b3] file:bg-[rgba(139, 92, 246,0.2)] file:text-
[#ff6b6b] file:border-0 file:rounded file:px-3 file:py-1 file:mr-3 file:cursor-pointer file:text-sm"
    />
  </div>
</div>

{/* Submit */}
{loading ? (
  <div className="flex items-center justify-center py-4">
    <div className="w-8 h-8 border-2 border-[#8b5cf6] border-t-transparent rounded-full animate-spin"></div>
  </div>
) : (
  <button
    type="submit"
    className="portal-btn w-full py-3 text-base rounded-lg"
  >
    Create Account
  </button>
)}

{/* Login link */}
<div className="text-center mt-6">
  <p className="text-[#b3b3b3]">
    Already have an account?{" "}
    <Link to="/login" className="text-white font-semibold hover:text-[#8b5cf6] transition-colors">
      Sign in
    </Link>
  </p>
</div>
</form>
</div>
</div>
);
};

export default Register;

```

Frontend\src\components\creator\Creator.jsx

```
import React from 'react'
import Navbar from '../pages/Navbar'
import amreshsir from './amreshsir.jpg'; // Import the local image
import ankit from './Ankit.jpg';
import ritik from './ritik.jpg';
import gaurav from './gaurav.jpg';

const Creator = () => {
  return (
    <div>
      <Navbar />
      <div className="flex flex-col items-center justify-center h-screen max-w-7xl mx-auto p-6">
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-center w-full">
          {/* Image Section */}
          <div className="flex justify-center">
            <img src={amreshsir} alt="Amresh Sir" className="h-80 object-cover rounded-lg shadow-md" />
          </div>
          {/* Text Section */}
          <div>
            <h2 className="text-2xl font-bold text-gray-800 mb-4">Dr. Amresh Kumar</h2>
            <p className="text-gray-600 mb-2">Dr. Amresh Kumar completed his <strong>B.Tech in Electronics and Communication Engineering (ECE)</strong> from <strong>Shivnand Singh Institution of Technology and Management, Aligarh</strong>, in <strong>2002</strong></p>
            <p className="text-gray-600 mb-2">After graduation, he joined <strong>Koderma Mines Institution</strong> as a <strong>part-time lecturer</strong>. He then pursued his <strong>M.Tech in Electrical Engineering</strong> from <strong>BIT Sindri, Dhanbad</strong></p>
            <p className="text-gray-600 mb-2">He was later selected as an <strong>Assistant Professor</strong> in the <strong>Electronics and Communication Engineering (ECE) Department</strong> at <strong>MIT Muzaffarpur</strong>. During his tenure there, he also completed his <strong>Ph.D. in Electrical Engineering</strong> from <strong>Bihar University, Muzaffarpur, in 2016</strong></p>
            <p className="text-gray-600">Following his Ph.D., he was transferred to <strong>Rashtrakavi Ramdhari Singh Dinkar College of Engineering (RRSDCE)</strong> as an <strong>Assistant Professor in the Electrical and Electronics Engineering (EEE) Department</strong>, where he continues to serve till date.</p>
          </div>
        </div>
      </div>
      <hr className="w-full border-gray-300 my-6" />
      <div className="text-center p-6">
        <h2 className="text-2xl font-bold text-gray-800 mb-6">Developers and Designers</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          {/* Developer 1 - Ankit Pathak */}
          <a href="https://ankitpathak.vercel.app/" target="_blank" rel="noopener noreferrer" className="block text-center">
            <img src={ankit} alt="Ankit Pathak" className="mx-auto rounded-lg shadow-md" />
            <h3 className="text-lg font-semibold text-gray-700 mt-2">Ankit Pathak</h3>
            <p className="text-gray-600 text-sm">Registration No: 21110125035</p>
            <p className="text-gray-600 text-sm">Full Stack Developer</p>
          </a>
          {/* Developer 2 - Ritik Shrivastava */}
          <a href="#" className="block text-center">
            <img src={ritik} alt="Ritik Shrivastava" className="mx-auto rounded-lg shadow-md" />
            <h3 className="text-lg font-semibold text-gray-700 mt-2">Ritik Shrivastava</h3>
            <p className="text-gray-600 text-sm">Registration No: 21110125043</p>
            <p className="text-gray-600 text-sm">UI/UX Designer</p>
          </a>
          {/* Developer 3 - Gaurav Kumar */}
          <a href="#" className="block text-center">
            <img src={gaurav} alt="Gaurav Kumar" className="mx-auto rounded-lg shadow-md" />
            <h3 className="text-lg font-semibold text-gray-700 mt-2">Gaurav Kumar</h3>
            <p className="text-gray-600 text-sm">Registration No: 21110125023</p>
            <p className="text-gray-600 text-sm">Research</p>
          </a>
        </div>
      </div>
    </div>
  )
}
```

```
export default Creator
```

Frontend\src\components\pages\AppliedJob.jsx

```

import React from "react";
import {
  Table,
  TableBody,
  TableCaption,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "../ui/table";
import { Badge } from "../ui/badge";
import { useSelector } from "react-redux";

const AppliedJob = () => {
  const { allAppliedJobs } = useSelector((store) => store.job);
  return (
    <div>
      <Table>
        <TableCaption>Recent Applied Jobs</TableCaption>
        <TableHeader>
          <TableRow>
            <TableHead>Date</TableHead>
            <TableHead>Job Title</TableHead>
            <TableHead>Company</TableHead>
            <TableHead className="text-right">Status</TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {allAppliedJobs.length <= 0 ? (
            <span>You have not applied any job yet. </span>
          ) : (
            allAppliedJobs.map((appliedJob) => (
              <TableRow key={appliedJob._id}>
                <TableCell>{appliedJob?.createdAt.split("T")[0]}</TableCell>
                <TableCell>{appliedJob.job?.title}</TableCell>
                <TableCell>{appliedJob.job?.company.name}</TableCell>
                <TableCell className="text-right">
                  <Badge
                    className={` ${
                      appliedJob?.status === "rejected"
                        ? "bg-red-500"
                        : appliedJob?.status === "accepted"
                        ? "bg-green-600"
                        : "bg-gray-500"
                    }`}
                  />
                </TableCell>
              </TableRow>
            ))
          )}
        </TableBody>
      </Table>
    </div>
  );
};

export default AppliedJob;

```

Frontend\src\components\pages\Browse.jsx

```

import React, { useEffect } from "react";
import Navbar from "../Navbar";
import InternshipCard from "../InternshipCard";
import { useDispatch, useSelector } from "react-redux";
import { setSearchQuery } from "@redux/jobSlice";
import useGetAllJobs from "@hooks/useGetAllJobs";

const Browse = () => {
  useGetAllJobs();
  const { allJobs } = useSelector((store) => store.job);
  const dispatch = useDispatch();
  useEffect(() => {
    return () => {
      dispatch(setSearchQuery(""));
    };
  }, []);
  return (
    <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
      <Navbar />
      <div className="pt-20 max-w-7xl mx-auto px-4">
        <div className="mb-8 animate-fade-in-up">
          <h1 className="text-3xl font-bold text-white mb-2">
            Search Results
          </h1>
          <p className="text-[#b3b3b3]">
            <span style={{ color: '#00d4ff' }}>{allJobs.length}</span> internships found
          </p>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-5 stagger-children pb-10">
          {allJobs.map((job) => {
            return <InternshipCard key={job._id} job={job} />;
          })}
        </div>
      </div>
    </div>
  );
};

export default Browse;

```

Frontend\src\components\pages\Categories.jsx

```

import React from "react";
import { useDispatch } from "react-redux";
import { useNavigate } from "react-router-dom";
import { setSearchchedQuery } from "@redux/jobSlice";

const Category = [
  "Frontend Developer",
  "Backend Developer",
  "Full Stack Developer",
  "Mern Developer",
  "Data Scientist",
  "DevOps Engineer",
  "Machine Learning Engineer",
  "AI Engineer",
  "Cybersecurity",
  "Product Manager",
  "UX/UI Designer",
  "Graphics Designer",
  "Video Editor",
];

const Categories = () => {
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const searchjobHandler = (query) => {
    dispatch(setSearchedQuery(query));
    navigate("/browse");
  };

  return (
    <div className="py-12 px-4" style={{ background: 'var(--portal-black)' }}>
      <div className="max-w-7xl mx-auto">
        <h2 className="section-title mb-10">Explore Categories</h2>

        <div className="portal-row stagger-children" style={{ paddingBottom: '10px' }}>
          {Category.map((category, index) => (
            <button
              key={index}
              onClick={() => searchjobHandler(category)}
              className="category-chip"
            >
              {category}
            </button>
          ))}
        </div>
      </div>
    </div>
  );
};

export default Categories;

```

Frontend\src\components\pages\EditProfileModal.jsx

```

import React, { useState } from "react";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "../ui/dialog";
import { Label } from "../ui/label";
import { Button } from "../ui/button";
import { useDispatch, useSelector } from "react-redux";
import axios from "axios";
import { toast } from "sonner";
import { USER_API_ENDPOINT } from "@utils/data";
import { setUser } from "@redux/authSlice";
import { Loader2 } from "lucide-react";

const EditProfileModal = ({ open, setOpen }) => {
  const [loading, setLoading] = useState(false);
  const { user } = useSelector((store) => store.auth);

  const [input, setInput] = useState({
    fullname: user?.fullname, // Corrected from fullnamenname to fullname
    email: user?.email,
    phoneNumber: user?.phoneNumber,
    bio: user?.profile?.bio,
    skills: user?.profile?.skills?.map((skill) => skill),
    file: user?.profile?.resume,
  });

  const dispatch = useDispatch();

```

```

const changeEventHandler = (e) => {
  setInput({ ...input, [e.target.name]: e.target.value });
};

const handleFileChange = async (e) => {
  e.preventDefault();
  const formData = new FormData();
  formData.append("fullname", input.fullname);
  formData.append("email", input.email);
  formData.append("phoneNumber", input.phoneNumber);
  formData.append("bio", input.bio);
  formData.append("skills", input.skills);

  if (input.file) {
    formData.append("file", input.file);
  }

  try {
    setLoading(true);
    const res = await axios.post(
      `${USER_API_ENDPOINT}/profile/update`,
      formData,
      {
        headers: {
          "Content-Type": "multipart/form-data",
        },
        withCredentials: true,
      }
    );
    if (res.data.success) {
      // dispatch(setUser(res.data.user));
      dispatch(setUser({ ...res.data.user, skills: input.skills }));
      toast.success(res.data.message);
    }
  } catch (error) {
    console.log(error);
    toast.error(error.response.data.message);
  } finally {
    setLoading(false);
  }
  setOpen(false);

  console.log(input);
};

const FileChangehandler = (e) => {
  const file = e.target.files?.[0];
  setInput({ ...input, file });
};

return (
  <div>
    <Dialog open={open}>
      <DialogContent
        className="sm:max-w-[500px]"
        onInteractOutside={() => setOpen(false)}
      >
        <DialogHeader>
          <DialogTitle>Edit Profile</DialogTitle>
        </DialogHeader>
        </* Form for editing profile */>
        <form onSubmit={handleFileChange}>
          <div className="grid gap-4 py-4">
            <div className="grid grid-cols-4 items-center gap-4">
              <Label htmlFor="name" className="text-right">
                Name
              </Label>
              <input
                type="text"
                id="name"
                value={input.fullname}
                name="name"
                onChange={changeEventHandler}
                className="col-span-3 border border-gray-300 rounded-md p-2"
              />
            </div>
            <div className="grid grid-cols-4 items-center gap-4">
              <Label htmlFor="email" className="text-right">
                Email
              </Label>
              <input
                type="email"
                id="email"
                value={input.email}
                name="email"
                onChange={changeEventHandler}

```



```

        className="col-span-3 border border-gray-300 rounded-md p-2"
      />
    </div>
    <div className="grid grid-cols-4 items-center gap-4">
      <Label htmlFor="phone" className="text-right">
        Phone
      </Label>
      <input
        type="tel"
        id="phone"
        value={input.phoneNumber} // Ensure this is correctly set
        name="phoneNumber" // Ensure this matches the expected key
        onChange={changeEventHandler}
        className="col-span-3 border border-gray-300 rounded-md p-2"
      />
    </div>
    <div className="grid grid-cols-4 items-center gap-4">
      <Label htmlFor="bio" className="text-right">
        Bio
      </Label>
      <input
        type="bio"
        id="bio"
        value={input.bio}
        name="bio"
        onChange={changeEventHandler}
        className="col-span-3 border border-gray-300 rounded-md p-2"
      />
    </div>
    { /* skills */ }
    <div className="grid grid-cols-4 items-center gap-4">
      <Label htmlFor="skills" className="text-right">
        Skills
      </Label>
      <input
        id="skills"
        name="skills"
        value={input.skills}
        onChange={changeEventHandler}
        className="col-span-3 border border-gray-300 rounded-md p-2"
      />
    </div>
    { /* Resume file upload */ }
    <div className="grid grid-cols-4 items-center gap-4">
      <Label htmlFor="file" className="text-right">
        Resume
      </Label>
      <input
        type="file"
        id="file"
        name="file"
        accept="application/pdf"
        onChange={FileChangehandler}
        className="col-span-3 border border-gray-300 rounded-md p-2"
      />
    </div>
  </div>
  </div>

  <DialogFooter>
    {loading ? (
      <Button className="w-full my-4">
        { " " }
        <Loader2 className="mr-2 h-4 w-4 animate-spin" /> Please wait{ " " }
      </Button>
    ) : (
      <Button type="submit" className="w-full my-4">
        Save
      </Button>
    ) }
  </DialogFooter>
</form>
</DialogContent>
</Dialog>
</div>
);
};
export default EditProfileModal;

```

Frontend\src\components\pages\Filtercard.jsx

```

import React, { useEffect, useState } from "react";
import { RadioGroup, RadioGroupItem } from "../../ui/radio-group";
import { useDispatch } from "react-redux";
import { setSearchchedQuery } from "@redux/jobSlice";

```

```

const filterData = [
  {
    filterType: "Location",
    array: [
      "Delhi",
      "Mumbai",
      "Kolhapur",
      "Pune",
      "Bangalore",
      "Hyderabad",
      "Chennai",
      "Remote",
    ],
  },
  {
    filterType: "Technology",
    array: [
      "Mern",
      "React",
      "Data Scientist",
      "Fullstack",
      "Node",
      "Python",
      "Java",
      "frontend",
      "backend",
      "mobile",
      "desktop",
    ],
  },
  {
    filterType: "Experience",
    array: ["0-3 years", "3-5 years", "5-7 years", "7+ years"],
  },
  {
    filterType: "Salary",
    array: ["0-50k", "50k-100k", "100k-200k", "200k+"],
  },
];

const Filter = () => {
  const [selectedValue, setSelectedValue] = useState("");
  const handleChange = (value) => {
    setSelectedValue(value);
  };
  const dispatch = useDispatch();
  useEffect(() => {
    dispatch(setSearchedQuery(selectedValue));
  }, [selectedValue]);

  return (
    <div className="filter-sidebar">
      <h1 className="font-bold text-lg text-white mb-1">Filter Internships</h1>
      <div className="h-[2px] w-12 bg-[#8b5cf6] mb-4 rounded-full"></div>
      <RadioGroup value={selectedValue} onValueChange={handleChange}>
        {filterData.map((data, index) => (
          <div key={index} className="mb-4">
            <h2 className="font-semibold text-sm text-[#00d4ff] uppercase tracking-wider mb-2">
              {data.filterType}
            </h2>

            {data.array.map((item, indx) => {
              const itemId = `Id${index}-${indx}`;
              return (
                <div key={itemId} className="flex items-center space-x-2 my-1.5 group cursor-pointer">
                  <RadioGroupItem
                    value={item}
                    id={itemId}
                    className="border-[rgba(255,255,255,0.2)] text-[#8b5cf6] data-[state=checked]:bg-[#8b5cf6] data-[state=checked]:border-[#8b5cf6]"
                  >
                    <label
                      htmlFor={itemId}
                      className="text-sm text-[#b3b3b3] group-hover:text-white transition-colors cursor-pointer"
                    >
                      {item}
                    </label>
                  </div>
                </div>
              );
            })}
          </div>
        ))}
      </RadioGroup>
    </div>
  );
};

```

```
});  
  
export default Filter;
```

Frontend\src\components\pages\Footer.jsx

```
import React from "react";  
import { Link } from "react-router-dom";  
  
const Footer = () => {  
  return (  
    <div>  
      { /* Footer for the current page */ }  
      <div  
        style={{  
          textAlign: "center",  
          padding: "20px",  
          backgroundColor: "#f1f1f1",  
        }}  
      >  
        <p>© 2024 Sunfire Sensei. All rights reserved.</p>  
        <p>  
          Powered by <a href="https://github.com/ankitpathak62">Ankit Pathak</a>  
        </p>  
        <p>  
          <Link to={"/PrivacyPolicy"}>Privacy Policy </Link> |  
          <Link to={"/TermsOfService"}> Terms of Service</Link>  
        </p>  
      </div>  
    </div>  
  );  
};  
  
export default Footer;
```

Frontend\src\components\pages\Header.jsx

```
import React, { useState } from "react";  
import { Search } from "lucide-react";  
import { useDispatch } from "react-redux";  
import { setSearchQuery } from "@redux/jobSlice";  
import { useNavigate } from "react-router-dom";  
  
const Header = () => {  
  const [query, setQuery] = useState("");  
  const dispatch = useDispatch();  
  const navigate = useNavigate();  
  
  const searchJobHandler = () => {  
    dispatch(setSearchQuery(query));  
    navigate("/browse");  
  };  
  
  const handleKeyDown = (e) => {  
    if (e.key === "Enter") {  
      searchJobHandler();  
    }  
  };  
  
  return (  
    <div className="hero-gradient min-h-[85vh] flex items-center justify-center relative overflow-hidden">  
      { /* Animated background particles */ }  
      <div className="absolute inset-0 overflow-hidden pointer-events-none">  
        <div className="absolute top-20 left-10 w-72 h-72 bg-[#8b5cf6] rounded-full opacity-[0.03] blur-[100px]"></div>  
        <div className="absolute bottom-20 right-10 w-96 h-96 bg-[#00d4ff] rounded-full opacity-[0.04] blur-[120px]"></div>  
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[600px] h-[600px] bg-[#8b5cf6] rounded-full opacity-[0.02] blur-[150px]"></div>  
      </div>  
  
      <div className="text-center relative z-10 max-w-4xl mx-auto px-4">  
        { /* Tagline */ }  
        <div className="animate-fade-in-up mb-6">  
          <span className="inline-flex items-center gap-2 px-5 py-2 rounded-full text-sm font-medium" style={{  
            background: 'rgba(139, 92, 246, 0.12)',  
            border: '1px solid rgba(139, 92, 246, 0.25)',  
            color: '#ff6b6b'  
          }}>  
            <span className="w-2 h-2 rounded-full bg-[#8b5cf6] animate-pulse"></span>  
            Your Gateway to Top Internships  
          </span>  
        </div>  
      </div>  
    </div>  
  );  
};
```

```

    { /* Main heading */ }
<h1 className="text-5xl md:text-7xl font-bold leading-tight mb-6 animate-fade-in-up" style={{ animationDelay: '0.15s' }}>
  <span className="text-white">Discover Your</span>
  <br />
  <span className="text-white">Next </span>
  <span style={{
    background: 'linear-gradient(135deg, #8b5cf6, #ff6b6b)',
    WebkitBackgroundClip: 'text',
    WebkitTextFillColor: 'transparent',
    backgroundClip: 'text'
  }}>
    Internship
  </span>
</h1>

{ /* Subtitle */ }
<p className="text-lg md:text-xl text-[#b3b3b3] mb-10 max-w-2xl mx-auto animate-fade-in-up leading-relaxed" style={{
animationDelay: '0.3s' }}>
  Explore thousands of internship opportunities from top companies.
  <br className="hidden md:block" />
  Launch your career with the perfect role.
</p>

{ /* Search bar */ }
<div className="animate-fade-in-up" style={{ animationDelay: '0.45s' }}>
  <div className="flex w-full max-w-xl mx-auto search-glow rounded-full items-center overflow-hidden">
    <div className="pl-5 pr-2">
      <Search className="h-5 w-5 text-[#b3b3b3]" />
    </div>
    <input
      type="text"
      onChange={(e) => setQuery(e.target.value)}
      onKeyDown={handleKeyDown}
      placeholder="Search internships, companies, roles..."
      className="flex-1 bg-transparent text-white py-4 px-2 outline-none placeholder:text-[#666]"
    />
    <button
      onClick={searchJobHandler}
      className="portal-btn rounded-none rounded-r-full px-8 h-full py-4"
    >
      Search
    </button>
  </div>
</div>

{ /* Quick stats */ }
<div className="flex items-center justify-center gap-8 mt-12 animate-fade-in-up" style={{ animationDelay: '0.6s' }}>
  <div className="text-center">
    <div className="text-2xl font-bold text-white">10K</div>
    <div className="text-sm text-[#b3b3b3]">Internships</div>
  </div>
  <div className="w-px h-10 bg-[rgba(255,255,255,0.1)]"></div>
  <div className="text-center">
    <div className="text-2xl font-bold text-white">500+</div>
    <div className="text-sm text-[#b3b3b3]">Companies</div>
  </div>
  <div className="w-px h-10 bg-[rgba(255,255,255,0.1)]"></div>
  <div className="text-center">
    <div className="text-2xl font-bold text-white">50K+</div>
    <div className="text-sm text-[#b3b3b3]">Students</div>
  </div>
</div>
</div>
);
};
export default Header;

```

Frontend\src\components\pages\Home.jsx

```

import React, { useEffect } from "react";
import { useSelector } from "react-redux";
import Navbar from "../Navbar";
import Header from "../Header";
import Categories from "../Categories";
import LatestJobs from "../LatestJobs";
import useGetAllJobs from "@/hooks/useGetAllJobs";
import { useNavigate } from "react-router-dom";

const Home = () => {
  const { loading, error } = useGetAllJobs(); // Trigger data fetch
  const jobs = useSelector((state) => state.job.allJobs); // Access Redux state

  console.log("Jobs in Component:", { loading, error, jobs }); // Log to check state
  const { user } = useSelector((store) => store.auth);
  const navigate = useNavigate();

  useEffect(() => {
    if (user?.role === "Recruiter") {
      navigate("/admin/companies");
    }
  }, []);

  return (
    <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
      <Navbar />
      { /* Spacer for fixed navbar */ }
      <div className="pt-16">
        <Header />
        <Categories />
        {loading && (
          <div className="flex items-center justify-center py-20">
            <div className="w-8 h-8 border-2 border-[#8b5cf6] border-t-transparent rounded-full animate-spin"></div>
          </div>
        )}
        {error && (
          <div className="text-center py-10">
            <p className="text-[#b3b3b3]">Error: {error}</p>
          </div>
        )}
        {!loading && !error && <LatestJobs jobs={jobs} />}
      </div>
    </div>
  );
};

export default Home;

```

Frontend\src\components\pages\InternshipCard.jsx

```

import React from "react";
import { useNavigate } from "react-router-dom";
import { Avatar, AvatarImage } from "../ui/avatar";
import { Bookmark } from "lucide-react";
import { toast } from "sonner";

const Job1 = ({ job }) => {
  const navigate = useNavigate();

  const daysAgoFunction = (mongodbTime) => {
    const createdAt = new Date(mongodbTime);
    const currentTime = new Date();
    const timeDifference = currentTime - createdAt;
    return Math.floor(timeDifference / (1000 * 24 * 60 * 60));
  };

  return (
    <div
      onClick={() => navigate(`/description/${job?._id || job?.id}`)}
      className="portal-card p-5 cursor-pointer"
    >
      { /* Header row */ }
      <div className="flex items-center justify-between mb-3">
        <p className="text-xs text-[#666]">
          {daysAgoFunction(job?.createdAt) === 0
            ? "Today"
            : `${daysAgoFunction(job?.createdAt)} days ago`}
        </p>
        <button
          onClick={(e) => {
            e.stopPropagation();
            toast.success("Saved to bookmarks!");
          }}
          className="w-8 h-8 rounded-full flex items-center justify-center transition-all duration-300 hover:bg-[rgb(139, 92, 246, 0.15)]"
        >

```

```

        style={{ border: '1px solid rgba(255,255,255,0.1)' }}
      >
        <Bookmark size={14} className="text-[#b3b3b3] hover:text-[#8b5cf6]" />
      </button>
    </div>

    {/* Company info */}
    <div className="flex items-center gap-3 mb-4">
      <div className="p-1 rounded-lg" style={{ border: '1px solid rgba(255,255,255,0.1)' }}>
        <Avatar className="w-10 h-10">
          <AvatarImage src={job?.company?.logo} />
        </Avatar>
      </div>
      <div>
        <h3 className="font-medium text-white text-sm">{job?.company?.name}</h3>
        <p className="text-xs text-[#666]">India</p>
      </div>
    </div>

    {/* Title & description */}
    <h2 className="font-bold text-lg text-white mb-2">{job?.title}</h2>
    <p className="text-sm text-[#b3b3b3] mb-4 line-clamp-2">{job?.description}</p>

    {/* Badges */}
    <div className="flex flex-wrap items-center gap-2 mb-4">
      <span className="portal-badge badge-cyan">
        {job?.position} Positions
      </span>
      <span className="portal-badge badge-red">
        {job?.jobType}
      </span>
      <span className="portal-badge badge-purple">
        {job?.salary} LPA
      </span>
    </div>

    {/* Action buttons */}
    <div className="flex items-center gap-3">
      <button
        onClick={(e) => {
          e.stopPropagation();
          navigate(`/description/${job?._id || job?.id}`);
        }}
        className="portal-btn-outline text-sm py-2 px-5 flex-1"
      >
        Details
      </button>
      <button
        onClick={(e) => {
          e.stopPropagation();
          toast.success("Job saved successfully!");
        }}
        className="portal-btn text-sm py-2 px-5 flex-1"
      >
        Save
      </button>
    </div>
  </div>
);
};

export default Job1;

```

Frontend\src\components\pages\InternshipDetails.jsx

```

import React, { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { JOB_API_ENDPOINT, APPLICATION_API_ENDPOINT } from "@/utils/data";
import axios from "axios";
import { useDispatch, useSelector } from "react-redux";
import { setSingleJob } from "@redux/jobSlice";
import { toast } from "sonner";
import Navbar from "../Navbar";
import { MapPin, DollarSign, Clock, Users, Briefcase, Calendar } from "lucide-react";

const Description = () => {
  const params = useParams();
  const jobId = params.id;

  const { singleJob } = useSelector((store) => store.job);
  const dispatch = useDispatch();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const { user } = useSelector((store) => store.auth);

  const isInitiallyApplied =

```

```

    singleJob?.application?.some(
      (application) => application.applicant === user?._id
    ) || false;
const [isApplied, setIsApplied] = useState(isInitiallyApplied);

const applyJobHandler = async () => {
  try {
    const res = await axios.get(
      `${APPLICATION_API_ENDPOINT}/apply/${jobId}`,
      { withCredentials: true }
    );
    if (res.data.success) {
      setIsApplied(true);
      const updateSingleJob = {
        ...singleJob,
        applications: [...singleJob.applications, { applicant: user?._id }],
      };
      dispatch(setSingleJob(updateSingleJob));
      console.log(res.data);
      toast.success(res.data.message);
    }
  } catch (error) {
    console.log(error.message);
    toast.error(error.response.data.message);
  }
};

useEffect(() => {
  const fetchSingleJobs = async () => {
    dispatch(setSingleJob(null));
    setLoading(true);
    setError(null);
    try {
      const res = await axios.get(`${JOB_API_ENDPOINT}/get/${jobId}`, {
        withCredentials: true,
      });
      console.log("API Response:", res.data);
      if (res.data.status) {
        dispatch(setSingleJob(res.data.job));
        setIsApplied(
          res.data.job.applications.some(
            (application) => application.applicant === user?._id
          )
        );
      } else {
        setError("Failed to fetch jobs.");
      }
    } catch (error) {
      console.error("Fetch Error:", error);
      setError(error.message || "An error occurred.");
    } finally {
      setLoading(false);
    }
  };

  fetchSingleJobs();
}, [jobId, dispatch, user?._id]);

if (!singleJob) {
  return (
    <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }} className="flex items-center justify-center">
      <div className="w-8 h-8 border-2 border-[#8b5cf6] border-t-transparent rounded-full animate-spin"></div>
    </div>
  );
}

return (
  <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
    <Navbar />
    <div className="pt-24 pb-10 max-w-4xl mx-auto px-4">
      <div className="animate-fade-in-up">
        { /* Header card */ }
        <div className="glass-card p-8 mb-6">
          <div className="flex items-start justify-between flex-wrap gap-4">
            <div className="flex-1">
              <h1 className="font-bold text-3xl text-white mb-4">{singleJob?.title}</h1>
              <div className="flex flex-wrap gap-3">
                <span className="portal-badge badge-cyan">
                  {singleJob?.position} Open Positions
                </span>
                <span className="portal-badge badge-red">
                  {singleJob?.salary} LPA
                </span>
                <span className="portal-badge badge-purple">
                  {singleJob?.location}
                </span>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
);

```

```

        <span className="portal-badge badge-green">
          {singleJob?.jobType}
        </span>
      </div>
    </div>
    <button
      onClick={isApplied ? null : applyJobHandler}
      disabled={isApplied}
      className={`px-8 py-3 rounded-lg font-semibold text-white transition-all duration-300 ${
        isApplied
          ? "bg-[#333] cursor-not-allowed text-[#666]"
          : "portal-btn animate-pulse-glow"
      }`}
    >
      {isApplied ? "Already Applied" : "Apply Now"}
    </button>
  </div>
</div>

{/* Description */}
<div className="glass-card p-8 mb-6">
  <h2 className="section-title text-xl mb-6">About this Internship</h2>
  <p className="text-[#b3b3b3] leading-relaxed mt-4">
    {singleJob?.description}
  </p>
</div>

{/* Details grid */}
<div className="glass-card p-8">
  <h2 className="section-title text-xl mb-6">Details</h2>
  <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mt-4">
    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(0, 212, 255, 0.1)' }}>
        <Briefcase size={18} className="text-[#00d4ff]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Role</p>
        <p className="text-white font-medium">{singleJob?.position} Open Positions</p>
      </div>
    </div>

    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(139, 92, 246, 0.1)' }}>
        <MapPin size={18} className="text-[#8b5cf6]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Location</p>
        <p className="text-white font-medium">{singleJob?.location}</p>
      </div>
    </div>

    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(114, 9, 183, 0.1)' }}>
        <DollarSign size={18} className="text-[#c77dff]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Salary</p>
        <p className="text-white font-medium">{singleJob?.salary} LPA</p>
      </div>
    </div>

    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(0, 200, 83, 0.1)' }}>
        <Clock size={18} className="text-[#69f0ae]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Experience</p>
        <p className="text-white font-medium">{singleJob?.experienceLevel} Year</p>
      </div>
    </div>

    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(255, 193, 7, 0.1)' }}>
        <Users size={18} className="text-[#ffc107]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Total Applicants</p>
        <p className="text-white font-medium">{singleJob?.applications?.length}</p>
      </div>
    </div>
  </div>

```



```

    <div className="flex items-center gap-4">
      <div className="w-10 h-10 rounded-lg flex items-center justify-center" style={{ background: 'rgba(255, 255, 255,
0.05)' }}>
        <Calendar size={18} className="text-[#b3b3b3]" />
      </div>
      <div>
        <p className="text-xs text-[#666] uppercase tracking-wider">Posted On</p>
        <p className="text-white font-medium">{singleJob?.createdAt.split("T")[0]}</p>
      </div>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
);
};

export default Description;

```

Frontend\src\components\pages\InternshipListings.jsx

```

import React, { useEffect, useState } from "react";
import Navbar from "../Navbar";
import FilterCard from "../Filtercard";
import Job1 from "../InternshipCard";
import { useSelector } from "react-redux";
import { motion } from "framer-motion";

const Jobs = () => {
  const { allJobs, searchedQuery } = useSelector((store) => store.job);
  const [filterJobs, setFilterJobs] = useState(allJobs);

  useEffect(() => {
    // If no search query is provided, reset to all jobs
    if (!searchedQuery || searchedQuery.trim() === "") {
      setFilterJobs(allJobs);
      return;
    }

    // Filter based on the searched query across various fields (title, description, etc.)
    const filteredJobs = allJobs.filter((job) => {
      const query = searchedQuery.toLowerCase();
      return (
        job.title?.toLowerCase().includes(query) ||
        job.description?.toLowerCase().includes(query) ||
        job.location?.toLowerCase().includes(query) ||
        job.experience?.toLowerCase().includes(query) ||
        job.salary?.toLowerCase().includes(query)
      );
    });

    setFilterJobs(filteredJobs);
  }, [allJobs, searchedQuery]);

  return (
    <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
      <Navbar />
      <div className="pt-20 max-w-7xl mx-auto px-4">
        { /* Page header */ }
        <div className="mb-8 animate-fade-in-up">
          <h1 className="text-3xl font-bold text-white mb-2">
            Find <span style={{ color: '#8b5cf6' }}>Internships</span>
          </h1>
          <p className="text-[#b3b3b3]">
            {filterJobs.length} internship{filterJobs.length !== 1 ? 's' : ''} available
          </p>
        </div>

        <div className="flex gap-6">
          <div className="w-[260px] flex-shrink-0">
            <FilterCard />
          </div>

          {filterJobs.length <= 0 ? (
            <div className="flex-1 flex items-center justify-center py-20">
              <div className="text-center">
                <p className="text-2xl text-[#b3b3b3] mb-2">No internships found</p>
                <p className="text-[#666]">Try adjusting your filters</p>
              </div>
            </div>
          ) : (
            <div className="flex-1 h-[88vh] overflow-y-auto pb-5 pr-2">
              <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-5">
                {filterJobs.map((job) => (
                  <motion.div
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    exit={{ opacity: 0, y: -20 }}
                    transition={{ duration: 0.3 }}
                    key={job.id || job._id}>
                    <Job1 job={job} />
                  </motion.div>
                ))}
              </div>
            </div>
          )}
        </div>
      </div>
    </div>
  );
};

export default Jobs;

```

Frontend\src\components\pages\LatestJobs.jsx

```

import React from "react";
import JobCards from "../TrendingCard";
import { useSelector } from "react-redux";

const LatestJobs = () => {
  const allJobs = useSelector((state) => state.job?.allJobs || []); // Safely access allJobs

  return (
    <div className="py-12 px-4" style={{ background: 'var(--portal-black)' }}>
      <div className="max-w-7xl mx-auto">
        <h2 className="section-title mb-10">
          Trending <span style={{ color: '#8b5cf6' }}>Internships</span>
        </h2>

        {allJobs.length === 0 ? (
          <div className="text-center py-16">
            <p className="text-[#b3b3b3] text-lg">No internships available right now</p>
            <p className="text-[#666] text-sm mt-2">Check back soon for new opportunities</p>
          </div>
        ) : (
          <div className="portal-row stagger-children">
            {allJobs.slice(0, 8).map((job) =>
              job?._id ? (
                <JobCards key={job._id} job={job} />
              ) : null
            )}
          </div>
        )}
      </div>
    </div>
  );
};

export default LatestJobs;

```

Frontend\src\components\pages\Navbar.jsx

```

import React from "react";
import { Link, useNavigate } from "react-router-dom";
import { Popover, PopoverContent, PopoverTrigger } from "../ui/popover";
import { Avatar, AvatarImage } from "../ui/avatar";
import { Button } from "../ui/button";
import { Logout, User2 } from "lucide-react";
import { useDispatch, useSelector } from "react-redux";
import { toast } from "sonner";
import axios from "axios"; // Import axios
import { setUser } from "@redux/authSlice";
import { USER_API_ENDPOINT } from "@utils/data";

const Navbar = () => {
  const { user } = useSelector((store) => store.auth);
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const logoutHandler = async () => {
    try {
      const res = await axios.post(`${USER_API_ENDPOINT}/logout`, {}, {
        withCredentials: true,
      });
      if (res && res.data && res.data.success) {
        dispatch(setUser(null));
        navigate("/");
        toast.success(res.data.message);
      } else {
        console.error("Error logging out:", res.data);
      }
    } catch (error) {
      console.error("Axios error:", error);
      if (error.response) {
        console.error("Error response:", error.response.data);
      }
      toast.error("Error logging out. Please try again.");
    }
  };

  return (
    <nav className="fixed top-0 left-0 right-0 z-50" style={{
      background: 'rgba(20, 20, 20, 0.85)',
      backdropFilter: 'blur(20px)',
      WebkitBackdropFilter: 'blur(20px)',
      borderBottom: '1px solid rgba(255, 255, 255, 0.06)'
    }}>
      <div className="flex items-center justify-between mx-auto max-w-7xl h-16 px-4">
        <Link to="/" className="flex items-center gap-1 group">
          <h1 className="text-2xl font-bold tracking-tight">
            <span style={{ color: '#8b5cf6' }}>Internship</span>

```

```

    <span className="text-white ml-1">Portal</span>
  </h1>
</Link>
<div className="flex items-center gap-8">
  <ul className="flex font-medium items-center gap-6">
    {user && user.role === "Recruiter" ? (
      <>
        <li>
          <Link
            to={"/admin/companies"}
            className="text-[#b3b3b3] hover:text-white transition-colors duration-300 relative after:content-['']
after:absolute after:bottom-[-4px] after:left-0 after:w-0 after:h-[2px] after:bg-[#8b5cf6] after:transition-all after:duration-300
hover:after:w-full"
          >
            Companies
          </Link>
        </li>
        <li>
          <Link
            to={"/admin/jobs"}
            className="text-[#b3b3b3] hover:text-white transition-colors duration-300 relative after:content-['']
after:absolute after:bottom-[-4px] after:left-0 after:w-0 after:h-[2px] after:bg-[#8b5cf6] after:transition-all after:duration-300
hover:after:w-full"
          >
            Jobs
          </Link>
        </li>
      </>
    ) : (
      <>
        <li>
          <Link
            to={"/Home"}
            className="text-[#b3b3b3] hover:text-white transition-colors duration-300 relative after:content-['']
after:absolute after:bottom-[-4px] after:left-0 after:w-0 after:h-[2px] after:bg-[#8b5cf6] after:transition-all after:duration-300
hover:after:w-full"
          >
            Home
          </Link>
        </li>
        <li>
          <Link
            to={"/Browse"}
            className="text-[#b3b3b3] hover:text-white transition-colors duration-300 relative after:content-['']
after:absolute after:bottom-[-4px] after:left-0 after:w-0 after:h-[2px] after:bg-[#8b5cf6] after:transition-all after:duration-300
hover:after:w-full"
          >
            Browse
          </Link>
        </li>
        <li>
          <Link
            to={"/Jobs"}
            className="text-[#b3b3b3] hover:text-white transition-colors duration-300 relative after:content-['']
after:absolute after:bottom-[-4px] after:left-0 after:w-0 after:h-[2px] after:bg-[#8b5cf6] after:transition-all after:duration-300
hover:after:w-full"
          >
            Internships
          </Link>
        </li>
      </>
    )}
  </ul>
  {!user ? (
    <div className="flex items-center gap-3">
      <Link to={"/login"} className="portal-btn-outline text-sm py-2 px-5 inline-block">
        Login
      </Link>
      <Link to={"/register"} className="portal-btn text-sm py-2 px-5 inline-block">
        Register
      </Link>
    </div>
  ) : (
    <Popover>
      <PopoverTrigger asChild>
        <Avatar className="cursor-pointer ring-2 ring-transparent hover:ring-[#8b5cf6] transition-all duration-300">
          <AvatarImage
            src={user?.profile?.profilePhoto}
            alt="@shadcn"
          />
        </Avatar>
      </PopoverTrigger>
      <PopoverContent className="w-80 glass-card border-0" style={{ background: 'rgba(26, 26, 46, 0.95)', backdropFilter:
'blur(20px)' }}>
        <div className="flex items-center gap-4 space-y-2">
          <Avatar className="cursor-pointer">

```

```

        <AvatarImage
          src={user?.profile?.profilePhoto}
          alt="@shadcn"
        />
      </Avatar>
    <div>
      <h3 className="font-medium text-white">{user?.fullname}</h3>
      <p className="text-sm text-[#b3b3b3]">
        {user?.profile?.bio}
      </p>
    </div>
  </div>

  <div className="flex flex-col my-2 text-[#b3b3b3]">
    {user && user.role === "Student" && (
      <div className="flex w-fit items-center gap-2 cursor-pointer hover:text-white transition-colors">
        <User2 size={16} />
        <Button variant="link" className="text-[#b3b3b3] hover:text-white p-0">
          <Link to={"/Profile"}>Profile</Link>
        </Button>
      </div>
    )}

    <div className="flex w-fit items-center gap-2 cursor-pointer hover:text-[#8b5cf6] transition-colors">
      <Logout size={16} />
      <Button onClick={logoutHandler} variant="link" className="text-[#b3b3b3] hover:text-[#8b5cf6] p-0">
        Logout
      </Button>
    </div>
  </div>
</PopoverContent>
</Popover>
  )}
</div>
</div>
</nav>
);
};

export default Navbar;

```

Frontend\src\components\pages\PrivacyPolicy.jsx

```

import React from "react";

const PrivacyPolicy = () => {
  return (
    <div>
      <h1>Privacy Policy for Internship Portal</h1>

      <h2>1. Introduction</h2>
      <p>
        This Privacy Policy outlines how we collect, use, and protect your
        information when you visit our internship portal website.
      </p>

      <h2>2. Information We Collect</h2>
      <ul>
        <li>
          <strong>Personal Information:</strong>
          <ul>
            <li>Name</li>
            <li>Email address</li>
            <li>Phone number</li>
            <li>Resume/CV</li>
          </ul>
        </li>
        <li>
          <strong>Usage Data:</strong>
          <ul>
            <li>IP address</li>
            <li>Browser type</li>
            <li>Pages visited</li>
            <li>Time spent on pages</li>
          </ul>
        </li>
      </ul>

      <h2>3. How We Use Your Information</h2>
      <ul>
        <li>To provide and maintain our services</li>
        <li>To notify you about changes to our services</li>
        <li>To allow you to participate in interactive features</li>
        <li>To provide customer support</li>
        <li>

```

```

        To gather analysis or valuable information so that we can improve our
        services
    </li>
    <li>To monitor the usage of our services</li>
    <li>To detect, prevent, and address technical issues</li>
</ul>

<h2>4. Data Security</h2>
<p>
    We take the security of your personal information seriously and
    implement appropriate technical and organizational measures to protect
    it.
</p>

<h2>5. Sharing Your Information</h2>
<p>
    We do not sell or rent your personal information to third parties. We
    may share your information with:
</p>
<ul>
    <li>Service providers who assist us in operating our website</li>
    <li>Law enforcement agencies if required by law</li>
</ul>

<h2>6. Your Rights</h2>
<p>You have the right to:</p>
<ul>
    <li>Access your personal information</li>
    <li>Request correction of your personal information</li>
    <li>Request deletion of your personal information</li>
</ul>

<h2>7. Changes to This Privacy Policy</h2>
<p>
    We may update our Privacy Policy from time to time. We will notify you
    of any changes by posting the new Privacy Policy on this page.
</p>

<h2>8. Contact Us</h2>
<p>
    If you have any questions about this Privacy Policy, please contact us
    at [your email address].
</p>
</div>
);
};

export default PrivacyPolicy;

```

Frontend\src\components\pages\Profile.jsx

```

import React, { useState } from "react";
import Navbar from "../Navbar";
import { Avatar, AvatarImage } from "../ui/avatar";
import { Contact, Mail, Pen, Download, CheckCircle2, AlertTriangle, Trophy } from "lucide-react";
import { Badge } from "../ui/badge";
import AppliedJob from "../AppliedJob";
import EditProfileModal from "../EditProfileModal";
import { useSelector } from "react-redux";
import useGetAppliedJobs from "@/hooks/useGetAllAppliedJobs";

const isResume = true;
const Profile = () => {
    useGetAppliedJobs();
    const [open, setOpen] = useState(false);
    const { user } = useSelector((store) => store.auth);
    return (
        <div style={{ background: 'var(--portal-black)', minHeight: '100vh' }}>
            <Navbar />
            <div className="pt-24 pb-10 px-4">
                <div className="flex justify-between items-start">
                    <div className="flex items-center gap-5">
                        <Avatar className="cursor-pointer h-24 w-24 ring-2 ring-[rgb(139, 92, 246,0.3)]">
                            <AvatarImage
                                src={user?.profile?.profilePhoto}
                                alt="profile"
                            />
                        </Avatar>
                    </div>
                    <div>
                        <h1 className="font-bold text-2xl text-white">{user?.fullname}</h1>
                        <p className="text-[#b3b3b3] mt-1">{user?.profile?.bio}</p>
                    </div>
                </div>
            </div>
        </div>
    );
};

```

```

</div>
<button
  onClick={() => setOpen(true)}
  className="portal-btn-outline py-2 px-4 text-sm flex items-center gap-2"
>
  <Pen size={14} />
  Edit
</button>
</div>

{/* Contact info */}
<div className="my-6 space-y-3">
  <div className="flex items-center gap-3">
    <div className="w-8 h-8 rounded-lg flex items-center justify-center" style={{ background: 'rgba(0, 212, 255, 0.1)' }}>
      <Mail size={14} className="text-[#00d4ff]" />
    </div>
    <a href={`mailto:${user?.email}`} className="text-[#b3b3b3] hover:text-white transition-colors">
      {user?.email}
    </a>
  </div>
  <div className="flex items-center gap-3">
    <div className="w-8 h-8 rounded-lg flex items-center justify-center" style={{ background: 'rgba(139, 92, 246, 0.1)' }}>
      <Contact size={14} className="text-[#8b5cf6]" />
    </div>
    <a href={`tel:${user?.phoneNumber}`} className="text-[#b3b3b3] hover:text-white transition-colors">
      {user?.phoneNumber}
    </a>
  </div>
</div>

{/* Skills */}
<div className="mb-6">
  <h3 className="text-sm text-[#666] uppercase tracking-wider mb-3">Skills</h3>
  <div className="flex flex-wrap items-center gap-2">
    {user?.profile?.skills.length !== 0 ? (
      user?.profile?.skills.map((item, index) => (
        <span key={index} className="portal-badge badge-cyan">
          {item}
        </span>
      ))
    ) : (
      <span className="text-[#666]">No skills added</span>
    )}
  </div>
</div>

{/* Resume */}
<div>
  <h3 className="text-sm text-[#666] uppercase tracking-wider mb-3">Resume</h3>
  {isResume ? (
    <a
      target="_blank"
      href={user?.profile?.resume}
      className="inline-flex items-center gap-2 text-[#00d4ff] hover:text-white transition-colors"
    >
      <Download size={14} />
      {user?.profile?.resumeOriginalName || 'Download Resume'}
    </a>
  ) : (
    <span className="text-[#666]">No Resume Found</span>
  )}
</div>

{/* AI Resume Analysis */}
{user?.profile?.resumeAnalysis && (
  <div className="mt-8 pt-6 border-t border-[rgba(255,255,255,0.1)]">
    <div className="flex items-center gap-3 mb-4">
      <Trophy className="text-[#8b5cf6]" size={20} />
      <h3 className="text-lg font-bold text-white">AI Resume Insights</h3>
      <span className="ml-auto text-sm font-semibold px-3 py-1 rounded-full" style={{ background: 'rgba(139, 92, 246, 0.2)', color: '#c77dff' }}>
        Score: {user.profile.resumeAnalysis.score}/100
      </span>
    </div>
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
      <div>
        <h4 className="text-sm text-[#666] uppercase tracking-wider mb-3">Strengths</h4>
        <ul className="space-y-2">
          {user.profile.resumeAnalysis.strengths?.map((strength, i) => (
            <li key={i} className="flex items-start gap-2 text-sm text-[#b3b3b3]">
              <CheckCircle2 size={16} className="text-[#69f0ae] mt-0.5 flex-shrink-0" />
              <span>{strength}</span>
            </li>
          ))}
        </ul>
      </div>
    </div>
  </div>
)

```

```

    </div>
    <div>
      <h4 className="text-sm text-[#666] uppercase tracking-wider mb-3">Areas for Improvement</h4>
      <ul className="space-y-2">
        {user.profile.resumeAnalysis.improvements?.map((improvement, i) => (
          <li key={i} className="flex items-start gap-2 text-sm text-[#b3b3b3]">
            <AlertTriangle size={16} className="text-[#ffc107] mt-0.5 flex-shrink-0" />
            <span>{improvement}</span>
          </li>
        ))}
      </ul>
    </div>
  </div>
</div>
)
</div>

{/* Applied Internships */}
<div className="max-w-4xl mx-auto mt-8 animate-fade-in-up" style={{ animationDelay: '0.2s' }}>
  <h2 className="section-title mb-6">Applied Internships</h2>
  <div className="glass-card p-6 mt-4">
    <AppliedJob />
  </div>
</div>
</div>

{/* Edit Profile Modal */}
<EditProfileModal open={open} setOpen={setOpen} />
</div>
);
};

export default Profile;

```

```
Frontend\src\components\pages\TermsOfService.jsx
```



```

import React from "react";

const TermsOfService = () => {
  return (
    <div>
      <h1>Terms and Conditions</h1>

      <h2>1. Introduction</h2>
      <p>
        Welcome to [Your Website Name]. These Terms and Conditions govern your
        use of our website located at [Your Website URL]. By accessing or using
        our website, you agree to comply with these terms.
      </p>

      <h2>2. Acceptance of Terms</h2>
      <p>
        By using our website, you confirm that you accept these Terms and
        Conditions and that you agree to comply with them. If you do not agree
        with any part of these terms, you must not use our website.
      </p>

      <h2>3. Changes to Terms</h2>
      <p>
        We reserve the right to modify these Terms and Conditions at any time.
        Any changes will be effective immediately upon posting on this page.
        Your continued use of the website after any changes constitutes your
        acceptance of the new Terms and Conditions.
      </p>

      <h2>4. User Responsibilities</h2>
      <p>
        You agree to use the website only for lawful purposes and in a way that
        does not infringe the rights of, restrict, or inhibit anyone else's use
        and enjoyment of the website.
      </p>

      <h2>5. Intellectual Property</h2>
      <p>
        All content, trademarks, and other intellectual property on the website
        are owned by or licensed to [Your Website Name]. You may not reproduce,
        distribute, or create derivative works from any content without our
        express written permission.
      </p>

      <h2>6. Limitation of Liability</h2>
      <p>
        To the fullest extent permitted by law, [Your Website Name] shall not be
        liable for any direct, indirect, incidental, special, consequential, or
        punitive damages arising from your use of the website.
      </p>

      <h2>7. Governing Law</h2>
      <p>
        These Terms and Conditions shall be governed by and construed in
        accordance with the laws of [Your Jurisdiction]. Any disputes arising in
        connection with these terms shall be subject to the exclusive
        jurisdiction of the courts of [Your Jurisdiction].
      </p>

      <h2>8. Contact Information</h2>
      <p>
        If you have any questions about these Terms and Conditions, please
        contact us at [Your Contact Information].
      </p>
    </div>
  );
};

export default TermsOfService;

```

Frontend\src\components\pages\TrendingCard.jsx

```

import React from "react";
import { useNavigate } from "react-router-dom";

const JobCards = ({ job }) => {
  const navigate = useNavigate();

  return (
    <div
      onClick={() => navigate(`/description/${job?._id || job?.id}`)}
      className="portal-card p-5 w-[300px] min-w-[300px] cursor-pointer"
    >
      {/* Company header */}
      <div className="flex items-center gap-3 mb-4">
        {job?.company?.logo ? (
          <img
            src={job.company.logo}
            alt={job.company.name}
            className="w-10 h-10 rounded-lg object-cover"
            style={{ border: '1px solid rgba(255,255,255,0.1)' }}
          />
        ) : (
          <div className="w-10 h-10 rounded-lg flex items-center justify-center text-sm font-bold"
            style={{ background: 'rgba(139, 92, 246, 0.2)', color: '#8b5cf6' }}>
            {job?.name?.charAt(0) || 'C'}
          </div>
        )}
      </div>
      <div>
        <h3 className="text-white font-medium text-sm">{job.name || job?.company?.name}</h3>
        <p className="text-[#666] text-xs">India</p>
      </div>
    </div>

    {/* Job title */}
    <h2 className="font-bold text-lg text-white mb-2 line-clamp-1">{job.title}</h2>

    {/* Description */}
    <p className="text-[#b3b3b3] text-sm mb-4 line-clamp-2 leading-relaxed">
      {job.description}
    </p>

    {/* Badges */}
    <div className="flex flex-wrap gap-2">
      <span className="portal-badge badge-cyan">
        {job.position} Positions
      </span>
      <span className="portal-badge badge-red">
        {job.salary} LPA
      </span>
      <span className="portal-badge badge-purple">
        {job.location}
      </span>
      {job.jobType && (
        <span className="portal-badge badge-green">
          {job.jobType}
        </span>
      )}
    </div>
  </div>
);
};

export default JobCards;

```

Frontend\src\components\ui\avatar.jsx

```

import * as React from "react"
import * as AvatarPrimitive from "@radix-ui/react-avatar"

import { cn } from "@lib/utils"

const Avatar = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Root
    ref={ref}
    className={cn("relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full", className)}
    {...props} />
))
Avatar.displayName = AvatarPrimitive.Root.displayName

const AvatarImage = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Image
    ref={ref}
    className={cn("aspect-square h-full w-full", className)}
    {...props} />
))
AvatarImage.displayName = AvatarPrimitive.Image.displayName

const AvatarFallback = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Fallback
    ref={ref}
    className={cn(
      "flex h-full w-full items-center justify-center rounded-full bg-muted",
      className
    )}
    {...props} />
))
AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName

export { Avatar, AvatarImage, AvatarFallback }

```

Frontend\src\components\ui\badge.jsx

```

import * as React from "react"
import { cva } from "class-variance-authority";

import { cn } from "@lib/utils"

const badgeVariants = cva(
  "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground shadow hover:bg-primary/80",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground hover:bg-secondary/80",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground shadow hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function Badge({
  className,
  variant,
  ...props
}) {
  return <div className={cn(badgeVariants({ variant }), className)} {...props} />;
}

export { Badge, badgeVariants }

```

Frontend\src\components\ui\button.jsx

```

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva } from "class-variance-authority";

import { cn } from "@lib/utils"

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transition-colors focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground shadow hover:bg-primary/90",
        destructive:
          "bg-destructive text-destructive-foreground shadow-sm hover:bg-destructive/90",
        outline:
          "border border-input bg-background shadow-sm hover:bg-accent hover:text-accent-foreground",
        secondary:
          "bg-secondary text-secondary-foreground shadow-sm hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2",
        sm: "h-8 rounded-md px-3 text-xs",
        lg: "h-10 rounded-md px-8",
        icon: "h-9 w-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
)

const Button = React.forwardRef(({ className, variant, size, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button"
  return (
    <Comp
      className={cn(buttonVariants({ variant, size, className }))}
      ref={ref}
      {...props} />
  );
})
Button.displayName = "Button"

export { Button, buttonVariants }

```

Frontend\src\components\ui\carousel.jsx

```

import * as React from "react"
import useEmblaCarousel from "embla-carousel-react";
import { ArrowLeft, ArrowRight } from "lucide-react"

import { cn } from "@lib/utils"
import { Button } from "@components/ui/button"

const CarouselContext = React.createContext(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

const Carousel = React.forwardRef((
  {
    orientation = "horizontal",
    opts,
    setApi,
    plugins,
    className,
    children,
    ...props
  },
  ref
) => {
  const [carouselRef, api] = useEmblaCarousel({

```

```

    ...opts,
    axis: orientation === "horizontal" ? "x" : "y",
  }, plugins)
const [canScrollPrev, setCanScrollPrev] = React.useState(false)
const [canScrollNext, setCanScrollNext] = React.useState(false)

const onSelect = React.useCallback((api) => {
  if (!api) {
    return
  }

  setCanScrollPrev(api.canScrollPrev())
  setCanScrollNext(api.canScrollNext())
}, [])

const scrollPrev = React.useCallback(() => {
  api?.scrollPrev()
}, [api])

const scrollNext = React.useCallback(() => {
  api?.scrollNext()
}, [api])

const handleKeyDown = React.useCallback((event) => {
  if (event.key === "ArrowLeft") {
    event.preventDefault()
    scrollPrev()
  } else if (event.key === "ArrowRight") {
    event.preventDefault()
    scrollNext()
  }
}, [scrollPrev, scrollNext])

React.useEffect(() => {
  if (!api || !setApi) {
    return
  }

  setApi(api)
}, [api, setApi])

React.useEffect(() => {
  if (!api) {
    return
  }

  onSelect(api)
  api.on("reInit", onSelect)
  api.on("select", onSelect)

  return () => {
    api?.off("select", onSelect)
  };
}, [api, onSelect])

return (
  <CarouselContext.Provider
    value={{
      carouselRef,
      api: api,
      opts,
      orientation:
        orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
      scrollPrev,
      scrollNext,
      canScrollPrev,
      canScrollNext,
    }}>
    <div
      ref={ref}
      onKeyDownCapture={handleKeyDown}
      className={cn("relative", className)}
      role="region"
      aria-roledescription="carousel"
      {...props}>
      {children}
    </div>
  </CarouselContext.Provider>
);
})
Carousel.displayName = "Carousel"

const CarouselContent = React.forwardRef(({ className, ...props }, ref) => {
  const { carouselRef, orientation } = useCarousel()

  return (

```

```

    (<div ref={carouselRef} className="overflow-hidden">
      <div
        ref={ref}
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props} />
      </div>
    );
  })
  CarouselContent.displayName = "CarouselContent"

  const CarouselItem = React.forwardRef(({ className, ...props }, ref) => {
    const { orientation } = useCarousel()

    return (
      <div
        ref={ref}
        role="group"
        aria-roledescription="slide"
        className={cn(
          "min-w-0 shrink-0 grow-0 basis-full",
          orientation === "horizontal" ? "pl-4" : "pt-4",
          className
        )}
        {...props} />
      );
    )
  })
  CarouselItem.displayName = "CarouselItem"

  const CarouselPrevious = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref) => {
    const { orientation, scrollPrev, canScrollPrev } = useCarousel()

    return (
      <Button
        ref={ref}
        variant={variant}
        size={size}
        className={cn("absolute h-8 w-8 rounded-full", orientation === "horizontal"
          ? "-left-12 top-1/2 -translate-y-1/2"
          : "-top-12 left-1/2 -translate-x-1/2 rotate-90", className)}
        disabled={!canScrollPrev}
        onClick={scrollPrev}
        {...props}>
        <ArrowLeft className="h-4 w-4" />
        <span className="sr-only">Previous slide</span>
      </Button>
    );
  })
  CarouselPrevious.displayName = "CarouselPrevious"

  const CarouselNext = React.forwardRef(({ className, variant = "outline", size = "icon", ...props }, ref) => {
    const { orientation, scrollNext, canScrollNext } = useCarousel()

    return (
      <Button
        ref={ref}
        variant={variant}
        size={size}
        className={cn("absolute h-8 w-8 rounded-full", orientation === "horizontal"
          ? "-right-12 top-1/2 -translate-y-1/2"
          : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90", className)}
        disabled={!canScrollNext}
        onClick={scrollNext}
        {...props}>
        <ArrowRight className="h-4 w-4" />
        <span className="sr-only">Next slide</span>
      </Button>
    );
  })
  CarouselNext.displayName = "CarouselNext"

  export { Carousel, CarouselContent, CarouselItem, CarouselPrevious, CarouselNext };

```

Frontend\src\components\ui\dialog.jsx

```

import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"

import { cn } from "@lib/Utils"

const Dialog = DialogPrimitive.Root

```

```

const DialogTrigger = DialogPrimitive.Trigger

const DialogPortal = DialogPrimitive.Portal

const DialogClose = DialogPrimitive.Close

const DialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className
    )}
    {...props} />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef(({ className, children, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background p-6
shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-
[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-
[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-
lg",
        className
      )}
      {...props}>
      {children}
    <DialogPrimitive.Close
      className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition-opacity hover:opacity-100
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none data-[state=open]:bg-accent data-
[state=open]:text-muted-foreground">
      <X className="h-4 w-4" />
      <span className="sr-only">Close</span>
    </DialogPrimitive.Close>
  </DialogPrimitive.Content>
</DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-1.5 text-center sm:text-left", className)}
    {...props} />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2", className)}
    {...props} />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight", className)}
    {...props} />
))
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog,
  DialogPortal,
  DialogOverlay,

```

```
DialogTrigger,  
DialogClose,  
DialogContent,  
DialogHeader,  
DialogFooter,  
DialogTitle,  
DialogDescription,  
}  
}
```

Frontend\src\components\ui\input.jsx

```
import * as React from "react"  
  
import { cn } from "@/lib/utils"  
  
const Input = React.forwardRef(({ className, type, ...props }, ref) => {  
  return (  
    <input  
      type={type}  
      className={cn(  
        "flex h-9 w-full rounded-md border border-input bg-transparent px-3 py-1 text-base shadow-sm transition-colors file:border-0  
file:bg-transparent file:text-sm file:font-medium file:text-foreground placeholder:text-muted-foreground focus-visible:outline-none  
focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",  
        className  
      )}  
      ref={ref}  
      {...props} />  
    );  
  })  
  Input.displayName = "Input"  
  
export { Input }
```

Frontend\src\components\ui\label.jsx

```
import * as React from "react"  
import * as LabelPrimitive from "@radix-ui/react-label"  
import { cva } from "class-variance-authority";  
  
import { cn } from "@/lib/utils"  
  
const labelVariants = cva(  
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70"  
)  
  
const Label = React.forwardRef(({ className, ...props }, ref) => {  
  <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)} {...props} />  
})  
Label.displayName = LabelPrimitive.Root.displayName  
  
export { Label }
```

Frontend\src\components\ui\popover.jsx


```

import * as React from "react"
import * as PopoverPrimitive from "@radix-ui/react-popover"

import { cn } from "@/lib/utils"

const Popover = PopoverPrimitive.Root

const PopoverTrigger = PopoverPrimitive.Trigger

const PopoverAnchor = PopoverPrimitive.Anchor

const PopoverContent = React.forwardRef(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
    </PopoverPrimitive.Portal>
  ))
PopoverContent.displayName = PopoverPrimitive.Content.displayName

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }

```

Frontend\src\components\ui\radio-group.jsx

```

import * as React from "react"
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
import { Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {
  return <RadioGroupPrimitive.Root className={cn("grid gap-2", className)} {...props} ref={ref} />;
})
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName

const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {
  return (
    <RadioGroupPrimitive.Item
      ref={ref}
      className={cn(
        "aspect-square h-4 w-4 rounded-full border border-primary text-primary shadow focus:outline-none focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      {...props}>
      <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
        <Circle className="h-3.5 w-3.5 fill-primary" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  );
})
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName

export { RadioGroup, RadioGroupItem }

```

Frontend\src\components\ui\select.jsx

```

import * as React from "react"
import * as SelectPrimitive from "@radix-ui/react-select"
import { Check, ChevronDown, ChevronUp } from "lucide-react"

import { cn } from "@/lib/utils"

const Select = SelectPrimitive.Root

const SelectGroup = SelectPrimitive.Group

const SelectValue = SelectPrimitive.Value

const SelectTrigger = React.forwardRef(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-9 w-full items-center justify-between whitespace-nowrap rounded-md border border-input bg-transparent px-3 py-2 text-sm"
    )}
    {...props}>
    {children}
  </SelectPrimitive.Trigger>

```

```

shadow-sm ring-offset-background placeholder:text-muted-foreground focus:outline-none focus:ring-1 focus:ring-ring disabled:cursor-
not-allowed disabled:opacity-50 [>span]:line-clamp-1",
  className
  )}
  {...props}>
  {children}
  <SelectPrimitive.Icon asChild>
    <ChevronDown className="h-4 w-4 opacity-50" />
  </SelectPrimitive.Icon>
</SelectPrimitive.Trigger>
))
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName

const SelectScrollUpButton = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollUpButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1", className)}
    {...props}>
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1", className)}
    {...props}>
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-96 min-w-[8rem] overflow-hidden rounded-md border bg-popover text-popover-foreground shadow-md data-
[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-
[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
        position === "popper" &&
        "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-[side=right]:translate-x-1 data-[side=top]:-
translate-y-1",
        className
      )}
    >
      position={position}
      {...props}>
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn("p-1", position === "popper" &&
          "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-width)]")}>
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", className)}
    {...props} />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-2 pr-8 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute right-2 flex h-3.5 w-3.5 items-center justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
)

```

```

))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName

export {
  Select,
  SelectGroup,
  SelectValue,
  SelectTrigger,
  SelectContent,
  SelectLabel,
  SelectItem,
  SelectSeparator,
  SelectScrollUpButton,
  SelectScrollDownButton,
}

```

Frontend\src\components\ui\sonner.jsx

```

import { useTheme } from "next-themes"
import { Toaster as Sonner } from "sonner"

const Toaster = ({
  ...props
}) => {
  const { theme = "system" } = useTheme()

  return (
    <Sonner
      theme={theme}
      className="toaster group"
      toastOptions={{
        classNames: {
          toast:
            "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground group-[.toaster]:border-border group-[.toaster]:shadow-lg",
          description: "group-[.toast]:text-muted-foreground",
          actionButton:
            "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
          cancelButton:
            "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
        },
      }}
      {...props} />
    );
}

export { Toaster }

```

Frontend\src\components\ui\table.jsx

```

import * as React from "react"

import { cn } from "@/lib/utils"

const Table = React.forwardRef(({ className, ...props }, ref) => (
  <div className="relative w-full overflow-auto">
    <table
      ref={ref}
      className={cn("w-full caption-bottom text-sm", className)}
      {...props} />
    </div>
  ))
Table.displayName = "Table"

const TableHeader = React.forwardRef(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
  ))
TableHeader.displayName = "TableHeader"

const TableBody = React.forwardRef(({ className, ...props }, ref) => (
  <tbody
    ref={ref}
    className={cn("[&_tr:last-child]:border-0", className)}
    {...props} />
  ))

```

```

TableBody.displayName = "TableBody"

const TableFooter = React.forwardRef(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn("border-t bg-muted/50 font-medium [>tr]:last:border-b-0", className)}
    {...props} />
))
TableFooter.displayName = "TableFooter"

const TableRow = React.forwardRef(({ className, ...props }, ref) => (
  <tr
    ref={ref}
    className={cn(
      "border-b transition-colors hover:bg-muted/50 data-[state=selected]:bg-muted",
      className
    )}
    {...props} />
))
TableRow.displayName = "TableRow"

const TableHead = React.forwardRef(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-10 px-2 text-left align-middle font-medium text-muted-foreground [&:has([role=checkbox])]:pr-0 [&
[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableHead.displayName = "TableHead"

const TableCell = React.forwardRef(({ className, ...props }, ref) => (
  <td
    ref={ref}
    className={cn(
      "p-2 align-middle [&:has([role=checkbox])]:pr-0 [&[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableCell.displayName = "TableCell"

const TableCaption = React.forwardRef(({ className, ...props }, ref) => (
  <caption
    ref={ref}
    className={cn("mt-4 text-sm text-muted-foreground", className)}
    {...props} />
))
TableCaption.displayName = "TableCaption"

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}

```

Frontend\src\hooks\useGetAllAppliedJobs.jsx

```

import { setAllAppliedJobs } from "@redux/jobSlice";
import { APPLICATION_API_ENDPOINT } from "@utils/data";
import axios from "axios";
import { useEffect } from "react";
import { useDispatch } from "react-redux";

const useGetAppliedJobs = () => {
  const dispatch = useDispatch();
  useEffect(() => {
    const fetchAppliedJobs = async () => {
      try {
        const res = await axios.get(`${APPLICATION_API_ENDPOINT}/get`, {
          withCredentials: true,
        });
        console.log("API Response:", res.data);
        if (res.data.success) {
          dispatch(setAllAppliedJobs(res.data.application));
        }
      } catch (error) {
        console.error(error);
      }
    };
    fetchAppliedJobs();
  }, [dispatch]);
  return null;
};

export default useGetAppliedJobs;

```

Frontend\src\hooks\usegetAllCompanies.jsx

```

import { setCompanies } from "@redux/companyslice";
import { COMPANY_API_ENDPOINT } from "@utils/data";
import axios from "axios";
import { useEffect, useState } from "react";
import { useDispatch } from "react-redux";

const useGetAllCompanies = () => {
  const dispatch = useDispatch();
  useEffect(() => {
    const fetchCompanies = async () => {
      try {
        const res = await axios.get(`${COMPANY_API_ENDPOINT}/get`, {
          withCredentials: true,
        });
        console.log("called");
        if (res.data.success) {
          dispatch(setCompanies(res.data.companies));
        }
      } catch (error) {
        console.log(error);
      }
    };
    fetchCompanies();
  }, []);
};

export default useGetAllCompanies;

```

Frontend\src\hooks\useGetAllJAdminobs.jsx

```

import { setAllAdminJobs } from "@redux/jobSlice";
import { JOB_API_ENDPOINT } from "@utils/data";
import axios from "axios";
import { useEffect, useState } from "react";
import { useDispatch } from "react-redux";

const useGetAllAdminJobs = () => {
  const dispatch = useDispatch();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchAllAdminJobs = async () => {
      setLoading(true);
      setError(null);
      try {
        const res = await axios.get(`${JOB_API_ENDPOINT}/getadminjobs`, {
          withCredentials: true,
        });
        console.log("API Response:", res.data);
        if (res.data.status) {
          // Updated success check
          dispatch(setAllAdminJobs(res.data.jobs));
        } else {
          setError("Failed to fetch jobs.");
        }
      } catch (error) {
        console.error("Fetch Error:", error);
        setError(error.message || "An error occurred.");
      } finally {
        setLoading(false);
      }
    };

    fetchAllAdminJobs();
  }, [dispatch]);

  return { loading, error };
};

export default useGetAllAdminJobs;

```

Frontend\src\hooks\useGetAllJobs.jsx

```

import { setAllJobs } from "@redux/jobSlice";
import { JOB_API_ENDPOINT } from "@utils/data";
import axios from "axios";
import { useEffect, useState } from "react";
import { useDispatch, useSelector } from "react-redux";

const useGetAllJobs = () => {
  const dispatch = useDispatch();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const { searchedQuery } = useSelector((store) => store.job);

  useEffect(() => {
    const fetchAllJobs = async () => {
      setLoading(true);
      setError(null);
      try {
        const res = await axios.get(
          `${JOB_API_ENDPOINT}/get?keyword=${searchedQuery}`,
          {
            withCredentials: true,
          }
        );
        console.log("API Response:", res.data);
        if (res.data.status) {
          // Updated success check
          dispatch(setAllJobs(res.data.jobs));
        } else {
          setError("Failed to fetch jobs.");
        }
      } catch (error) {
        console.error("Fetch Error:", error);
        setError(error.message || "An error occurred.");
      } finally {
        setLoading(false);
      }
    };

    fetchAllJobs();
  }, [dispatch]);

  return { loading, error };
};

export default useGetAllJobs;

```

Frontend\src\hooks\useGetCompanyById.jsx

```

import { setSingleCompany } from "@redux/companyslice";
import { COMPANY_API_ENDPOINT } from "@utils/data";
import axios from "axios";
import { useEffect } from "react";
import { useDispatch } from "react-redux";

const useGetCompanyById = (companyId) => {
  const dispatch = useDispatch();

  useEffect(() => {
    const fetchSingleCompany = async () => {
      try {
        const res = await axios.get(
          `${COMPANY_API_ENDPOINT}/get/${companyId}`,
          { withCredentials: true }
        );
        dispatch(setSingleCompany(res.data.company));
      } catch (error) {
        console.error("Error fetching company:", error);
      }
    };

    if (companyId) {
      fetchSingleCompany();
    }
  }, [companyId, dispatch]);
};

export default useGetCompanyById;

```

Frontend\src\lib\utils.js

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge"

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

Frontend\src\main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";
import { Toaster } from "./components/ui/sonner";
import { Provider } from "react-redux";
import { persistStore } from "redux-persist";

import store from "./redux/store";
import { PersistGate } from "redux-persist/integration/react";

const persistor = persistStore(store);

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <Provider store={store}>
      <PersistGate loading={null} persistor={persistor}>
        <App />
        <Toaster />
      </PersistGate>
    </Provider>
  </StrictMode>
);
```

Frontend\src\redux\applicationSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const applicationSlice = createSlice({
  name: 'application',
  initialState: {
    applicants: null,
  },
  reducers: {
    setAllApplicants: (state, action) => {
      state.applicants = action.payload;
    }
  }
});

export const {setAllApplicants} = applicationSlice.actions;
export default applicationSlice.reducer;
export const applicationReducer = applicationSlice.reducer;
```

Frontend\src\redux\authSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const authSlice = createSlice({
  name: "auth",
  initialState: {
    loading: false,
    user: null,
  },
  reducers: {
    setLoading: (state, action) => {
      state.loading = action.payload;
    },
    setUser: (state, action) => {
      state.user = action.payload;
    },
  },
});

// Export the actions
export const {setLoading, setUser} = authSlice.actions;

// Export the reducer
export default authSlice.reducer;

// Export the authSlice if needed
export const authSliceReducer = authSlice.reducer;
```


Frontend\src\redux\companyslice.js

```
import { createSlice } from "@reduxjs/toolkit";
const companySlice = createSlice({
  name: "company",
  initialState: {
    singleCompany: {}, // Initialize with an empty object
    companies: [],
    searchCompanyByText: "",
  },
  reducers: {
    setSingleCompany: (state, action) => {
      state.singleCompany = action.payload;
    },
    setCompanies: (state, action) => {
      state.companies = action.payload;
    },
    setSearchCompanyByText: (state, action) => {
      state.searchCompanyByText = action.payload;
    },
  },
});

export const { setSingleCompany, setCompanies, setSearchCompanyByText } = companySlice.actions;

export default companySlice.reducer;

export { companySlice };
```

Frontend\src\redux\jobSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  allJobs: [],
  allAdminJobs: [], // This will hold
  singleJob: null, // This will hold the job details when a user clicks on a job
  searchJobByText: "",
  allAppliedJobs: [], // This will hold
  searchedQuery: "",
};

const jobSlice = createSlice({
  name: "jobs",
  initialState,
  reducers: {
    setAllJobs(state, action) {
      state.allJobs = action.payload; // Update state with fetched jobs
    },
    setSingleJob(state, action) {
      state.singleJob = action.payload; // Update state with fetched job details
    },
    setAllAdminJobs(state, action) {
      state.allAdminJobs = action.payload; // Update state with fetched admin jobs
    },
    setSearchJobByText(state, action) {
      state.searchJobByText = action.payload;
    },
    setAllAppliedJobs(state, action) {
      state.allAppliedJobs = action.payload;
    },
    setSearchedQuery(state, action) {
      state.searchedQuery = action.payload;
    },
  },
});

export const {
  setAllJobs,
  setSingleJob,
  setAllAdminJobs,
  setSearchJobByText,
  setAllAppliedJobs,
  setSearchedQuery,
} = jobSlice.actions;
export default jobSlice.reducer;
```

Frontend\src\redux\store.js

```

import { combineReducers, configureStore } from "@reduxjs/toolkit";
import authReducer from "../authSlice";
import jobSlice from "../jobSlice";
import jobReducer from "../jobSlice";
import { createRoot } from "react-dom/client";
import { companySlice } from "../companyslice";
import companyReducer from "../companyslice";

import {
  persistStore,
  persistReducer,
  FLUSH,
  REHYDRATE,
  PAUSE,
  PERSIST,
  PURGE,
  REGISTER,
} from "redux-persist";
import storage from "redux-persist/lib/storage";
import applicationSlice from "../applicationSlice";
const persistConfig = {
  key: "root",
  version: 1,
  storage,
};

const rootReducer = combineReducers({
  auth: authReducer,
  job: jobSlice,
  company: companySlice,
  application: applicationSlice,
});

const persistedReducer = persistReducer(persistConfig, rootReducer);

const store = configureStore({
  reducer: persistedReducer,
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      serializableCheck: {
        ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE, REGISTER],
      },
    }),
});

export default store;

```

Frontend\src\utils\data.js

```

export const USER_API_ENDPOINT = "http://localhost:5011/api/user";
export const JOB_API_ENDPOINT = "http://localhost:5011/api/job";
export const APPLICATION_API_ENDPOINT = "http://localhost:5011/api/application";
export const COMPANY_API_ENDPOINT = "http://localhost:5011/api/company";

```

Backend\controllers\application.controller.js

```

import { Application } from "../models/application.model.js";
import { Job } from "../models/job.model.js";

export const applyJob = async (req, res) => {
  try {
    const userId = req.id;
    const jobId = req.params.id;
    if (!jobId) {
      return res
        .status(400)
        .json({ message: "Invalid job id", success: false });
    }
    // check if the user already has applied for this job
    const existingApplication = await Application.findOne({
      job: jobId,
      applicant: userId,
    });
    if (existingApplication) {
      return res.status(400).json({
        message: "You have already applied for this job",
        success: false,
      });
    }
    //check if the job exists or not
    const job = await Job.findById(jobId);
    if (!job) {
      return res.status(404).json({ message: "Job not found", success: false });
    }
  }
};

```

```

    }
    // create a new application

    const newApplication = await Application.create({
      job: jobId,
      applicant: userId,
    });
    job.applications.push(newApplication._id);
    await job.save();

    return res
      .status(201)
      .json({ message: "Application submitted", success: true });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server error", success: false });
  }
};

export const getAppliedJobs = async (req, res) => {
  try {
    const userId = req.id;
    const application = await Application.find({ applicant: userId })
      .sort({ createdAt: -1 })
      .populate({
        path: "job",
        options: { sort: { createdAt: -1 } },
        populate: { path: "company", options: { sort: { createdAt: -1 } } },
      });
    if (!application) {
      return res
        .status(404)
        .json({ message: "No applications found", success: false });
    }

    return res.status(200).json({ application, success: true });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server error", success: false });
  }
};

export const getApplicants = async (req, res) => {
  try {
    const jobId = req.params.id;
    const job = await Job.findById(jobId).populate({
      path: "applications",
      options: { sort: { createdAt: -1 } },
      populate: { path: "applicant", options: { sort: { createdAt: -1 } } },
    });
    if (!job) {
      return res.status(404).json({ message: "Job not found", success: false });
    }

    return res.status(200).json({ job, success: true });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server error", success: false });
  }
};

export const updateStatus = async (req, res) => {
  try {
    const { status } = req.body;
    const applicationId = req.params.id;
    if (!status) {
      return res.status(400).json({
        message: "status is required",
        success: false,
      });
    }

    // find the application by application id
    const application = await Application.findOne({ _id: applicationId });
    if (!application) {
      return res.status(404).json({
        message: "Application not found.",
        success: false,
      });
    }

    // update the status
    application.status = status.toLowerCase();
    await application.save();

    return res

```

```

        .status(200)
        .json({ message: "Application status updated", success: true });
    } catch (error) {
        console.error(error);
        res.status(500).json({ message: "Server error", success: false });
    }
};

```

Backend\controllers\company.controller.js

```

import { Company } from "../models/company.model.js";
import getDataUri from "../utils/datauri.js";
import cloudinary from '../utils/cloud.js';

export const registerCompany = async (req, res) => {
    try {
        const { companyName } = req.body;
        if (!companyName) {
            return res.status(401).json({
                message: "Company name is required",
                success: false,
            });
        }
        let company = await Company.findOne({ name: companyName });
        if (company) {
            return res.status(401).json({
                message: "Company already exists",
                success: false,
            });
        }
        company = await Company.create({
            name: companyName,
            userId: req.id,
        });
        return res.status(201).json({
            message: "Company registered successfully.",
            company,
            success: true,
        });
    } catch (error) {
        console.log(error);
    }
};

export const getAllCompanies = async (req, res) => {
    try {
        const userId = req.id; // loggedin user id
        const companies = await Company.find({ userId });
        if (!companies) {
            return res.status(404).json({ message: "No companies found" });
        }
        return res.status(200).json({
            companies,
            success: true,
        });
    } catch (error) {
        console.error(error);
    }
};

//get company by id
export const getCompanyById = async (req, res) => {
    try {
        const companyId = req.params.id;
        const company = await Company.findById(companyId);
        if (!company) {
            return res.status(404).json({ message: "Company not found" });
        }
        return res.status(200).json({ company, success: true });
    } catch (error) {
        console.error(error);
    }
};

//update company details
export const updateCompany = async (req, res) => {
    try {
        const { name, description, website, location } = req.body;
        const file = req.file;
        //cloudinary
        const fileUri = getDataUri(file);
        const cloudResponse = await cloudinary.uploader.upload(fileUri.content);
        const logo = cloudResponse.secure_url;
    }
};

```

```

    const updateData = { name, description, website, location, logo };

    const company = await Company.findByIdAndUpdate(req.params.id, updateData, {
      new: true,
    });
    if (!company) {
      return res.status(404).json({ message: "Company not found" });
    }
    return res.status(200).json({ message: "Company updated" });
  } catch (error) {
    console.error(error);
  }
};

```

Backend\controllers\job.controller.js

```

import { Job } from "../models/job.model.js";
//Admin job posting
export const postJob = async (req, res) => {
  try {
    const {
      title,
      description,
      requirements,
      salary,
      location,
      jobType,
      experience,
      position,
      companyId,
    } = req.body;
    const userId = req.id;

    if (
      !title ||
      !description ||
      !requirements ||
      !salary ||
      !location ||
      !jobType ||
      !experience ||
      !position ||
      !companyId
    ) {
      return res.status(400).json({
        message: "All fields are required",
        success: false,
      });
    }
    const job = await Job.create({
      title,
      description,
      requirements: requirements.split(","),
      salary: Number(salary),
      location,
      jobType,
      experienceLevel: experience,
      position,
      company: companyId,
      created_by: userId,
    });
    res.status(201).json({
      message: "Job posted successfully.",
      job,
      status: true,
    });
  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Server Error", status: false });
  }
};

//Users
export const getAllJobs = async (req, res) => {
  try {
    const keyword = req.query.keyword || "";
    const query = {
      $or: [
        { title: { $regex: keyword, $options: "i" } },
        { description: { $regex: keyword, $options: "i" } },
      ],
    };
  } catch (error) {
    console.error(error);
  }
  const jobs = await Job.find(query)
    .populate({
      path: "company",

```

```

    })
    .sort({ createdAt: -1 });

    if (!jobs) {
      return res.status(404).json({ message: "No jobs found", status: false });
    }
    return res.status(200).json({ jobs, status: true });
  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Server Error", status: false });
  }
};

//Users
export const getJobById = async (req, res) => {
  try {
    const jobId = req.params.id;
    const job = await Job.findById(jobId).populate({
      path: "applications",
    });
    if (!job) {
      return res.status(404).json({ message: "Job not found", status: false });
    }
    return res.status(200).json({ job, status: true });
  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Server Error", status: false });
  }
};

//Admin job created
export const getAdminJobs = async (req, res) => {
  try {
    const adminId = req.id;
    const jobs = await Job.find({ created_by: adminId }).populate({
      path: "company",
      sort: { createdAt: -1 },
    });
    if (!jobs) {
      return res.status(404).json({ message: "No jobs found", status: false });
    }
    return res.status(200).json({ jobs, status: true });
  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Server Error", status: false });
  }
};

```

Backend\controllers\user.controller.js

```

import { User } from "../models/user.model.js";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import getDataUri from "../utils/datauri.js";
import cloudinary from "../utils/cloud.js";
import { GoogleGenAI } from "@google/genai";
import pdfParse from "pdf-parse/lib/pdf-parse.js";

export const register = async (req, res) => {
  try {
    const { fullname, email, phoneNumber, password, adharcard, pancard, role } = req.body;

    if (!fullname || !email || !phoneNumber || !password || !role || !pancard || !adharcard) {
      return res.status(400).json({
        message: "Missing required fields",
        success: false,
      });
    }

    const user = await User.findOne({ email });
    if (user) {
      return res.status(400).json({
        message: "Email already exists",
        success: false,
      });
    }

    const existingAdharcard = await User.findOne({ adharcard });
    if (existingAdharcard) {
      return res.status(400).json({
        message: "Adhar number already exists",
        success: false,
      });
    }
  }
};

```

```

const existingPancard = await User.findOne({ pancard });
if (existingPancard) {
  return res.status(400).json({
    message: "Pan number already exists",
    success: false,
  });
}

const file = req.file;
let profilePhoto = "";
if (file) {
  const fileUri = getDataUri(file);
  const cloudResponse = await cloudinary.uploader.upload(fileUri.content);
  profilePhoto = cloudResponse.secure_url;
}

const hashedPassword = await bcrypt.hash(password, 10);

const newUser = new User({
  fullname,
  email,
  phoneNumber,
  adharcard,
  pancard,
  password: hashedPassword,
  role,
  profile: {
    profilePhoto: profilePhoto,
  },
});

await newUser.save();

return res.status(201).json({
  message: `Account created successfully for ${fullname}`,
  success: true,
});
} catch (error) {
  console.error(error);
  res.status(500).json({
    message: "Server Error registering user",
    success: false,
  });
}
};

export const login = async (req, res) => {
  try {
    const { email, password, role } = req.body;

    if (!email || !password || !role) {
      return res.status(400).json({
        message: "Missing required fields",
        success: false,
      });
    }

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(404).json({
        message: "Incorrect email or password",
        success: false,
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(400).json({
        message: "Incorrect email or password",
        success: false,
      });
    }

    if (user.role !== role) {
      return res.status(403).json({
        message: "You don't have the necessary role to access this resource",
        success: false,
      });
    }

    const tokenData = {
      userId: user._id,
    };
    const token = jwt.sign(tokenData, process.env.JWT_SECRET, {
      expiresIn: "1d",
    });
  }
};

```

```

});

const sanitizedUser = {
  _id: user._id,
  fullname: user.fullname,
  email: user.email,
  phoneNumber: user.phoneNumber,
  adharcard: user.adharcard,
  pancard: user.pancard,
  role: user.role,
  profile: user.profile,
};

return res
  .status(200)
  .cookie("token", token, {
    maxAge: 24 * 60 * 60 * 1000,
    httpOnly: true,
    sameSite: "Strict",
  })
  .json({
    message: `Welcome back ${user.fullname}`,
    user: sanitizedUser,
    success: true,
  });
} catch (error) {
  console.error(error);
  res.status(500).json({
    message: "Server Error login failed",
    success: false,
  });
}
};

export const logout = async (req, res) => {
  try {
    return res.status(200).cookie("token", "", { maxAge: 0 }).json({
      message: "Logged out successfully",
      success: true,
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({
      message: "Server Error logging out",
      success: false,
    });
  }
};

export const updateProfile = async (req, res) => {
  try {
    const { fullname, email, phoneNumber, bio, skills } = req.body;
    const file = req.file;

    const userId = req.id; // Assuming authentication middleware sets req.id
    const user = await User.findById(userId);

    if (!user) {
      return res.status(404).json({
        message: "User not found",
        success: false,
      });
    }

    if (fullname) user.fullname = fullname;
    if (email) user.email = email;
    if (phoneNumber) user.phoneNumber = phoneNumber;
    if (bio) user.profile.bio = bio;
    if (skills) user.profile.skills = skills.split(",");

    if (file) {
      const fileUri = getDataUri(file);
      const cloudResponse = await cloudinary.uploader.upload(fileUri.content);
      user.profile.resume = cloudResponse.secure_url;
      user.profile.resumeOriginalname = file.originalname;

      // AI Analysis
      try {
        if (process.env.GEMINI_API_KEY) {
          const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
          const pdfData = await pdfParse(file.buffer);
          const resumeText = pdfData.text;

          const prompt = `Analyze this resume text and provide a JSON response exactly in this format without any markdown wrappers or extra text: {"score": 85, "strengths": ["strength1", "strength2"], "improvements": ["improvement1", "improvement2"]}. Here is the resume text:\n\n${resumeText}`;

```



```

    const aiResponse = await ai.models.generateContent({
      model: "gemini-2.5-flash",
      contents: prompt,
    });

    let aiText = aiResponse.text;
    if (aiText.startsWith("`json`")) {
      aiText = aiText.replace(/`json/g, "").replace(/`/g, "").trim();
    } else if (aiText.startsWith("`")) {
      aiText = aiText.replace(/`/g, "").trim();
    }

    const analysis = JSON.parse(aiText);
    user.profile.resumeAnalysis = analysis;
  } else {
    console.warn("GEMINI_API_KEY is not set. Skipping AI Resume Analysis.");
  }
} catch (e) {
  console.error("AI Analysis failed:", e);
}

await user.save();

const updatedUser = {
  _id: user._id,
  fullname: user.fullname,
  email: user.email,
  phoneNumber: user.phoneNumber,
  role: user.role,
  profile: user.profile,
};

return res.status(200).json({
  message: "Profile updated successfully",
  user: updatedUser,
  success: true,
});
} catch (error) {
  console.error(error);
  res.status(500).json({
    message: "Server Error updating profile",
    success: false,
  });
}
};

```

```
// import { User } from "../models/user.model.js";
// import bcrypt from "bcryptjs";
// import jwt from "jsonwebtoken";
// import getDataUri from "../utils/datauri.js";
// import cloudinary from "../utils/cloud.js";

// export const register = async (req, res) => {
//   try {
//     const { fullname, email, phoneNumber, password, adharcard, pancard, role } =
//       req.body;

//     if (
//       !fullname ||
//       !email ||
//       !phoneNumber ||
//       !password ||
//       !role ||
//       !pancard ||
//       !adharcard
//     ) {
//       return res.status(404).json({
//         message: "Missing required fields",
//         success: false,
//       });
//     }
//     const file = req.file;
//     const fileUri = getDataUri(file);
//     const cloudResponse = await cloudinary.uploader.upload(fileUri.content);

//     const user = await User.findOne({ email });
//     if (user) {
//       return res.status(400).json({
//         message: "Email already exists",
//         success: false,
//       });
//     }
//     const user = await User.findOne({ adharcard });
//     if (adharcard) {
//       return res.status(400).json({
//         message: "Adhar number already exists",
//         success: false,
//       });
//     }
//   }
// }
```

```

//    }
//    const user = await User.findOne({ pancard });
//    if (pancard) {
//        return res.status(400).json({
//            message: "Pan number already exists",
//            success: false,
//        });
//    }
//    //convert passwords to hashes
//    const hashedPassword = await bcrypt.hash(password, 10);

//    const newUser = new User({
//        fullname,
//        email,
//        phoneNumber,
//        adharcard,
//        pancard,
//        password: hashedPassword,
//        role,
//        profile: {
//            profilePhoto: cloudResponse.secure_url,
//        },
//    });

//    await newUser.save();

//    return res.status(200).json({
//        message: `Account created successfully ${fullname}`,
//        success: true,
//    });
// } catch (error) {
//     console.error(error);
//     res.status(500).json({
//         message: "Server Error registering user",
//         success: false,
//     });
// }
// };

// export const login = async (req, res) => {
//     try {
//         const { email, password, role } = req.body;

//         if (!email || !password || !adharcard || !role) {
//             return res.status(404).json({
//                 message: "Missing required fields",
//                 success: false,
//             });
//         }
//         let user = await User.findOne({ email });
//         if (!user) {
//             return res.status(404).json({
//                 message: "Incorrect email or password",
//                 success: false,
//             });
//         }
//         let user = await User.findOne({ adharcard });
//         if (adharcard) {
//             return res.status(404).json({
//                 message: "Incorrect Adhar Number",
//                 success: false,
//             });
//         }
//         const isMatch = await bcrypt.compare(password, user.password);
//         if (!isMatch) {
//             return res.status(404).json({
//                 message: "Incorrect email or password",
//                 success: false,
//             });
//         }
//         //check role correctly or not
//         if (user.role !== role) {
//             return res.status(403).json({
//                 message: "You don't have the necessary role to access this resource",
//                 success: false,
//             });
//         }

//         //generate token
//         const tokenData = {
//             userId: user._id,
//         };
//         const token = await jwt.sign(tokenData, process.env.JWT_SECRET, {
//             expiresIn: "1d",
//         });

```

```

//      user = {
//        _id: user._id,
//        fullname: user.fullname,
//        email: user.email,
//        phoneNumber: user.phoneNumber,
//        adharcard: user.adharcard,
//        pancard: user.pancard,
//        role: user.role,
//        profile: user.profile,
//      };

//      return res
//        .status(200)
//        .cookie("token", token, {
//          maxAge: 1 * 24 * 60 * 60 * 1000,
//          httpOnly: true,
//          sameSite: "Strict",
//        })
//        .json({
//          message: `Welcome back ${user.fullname}`,
//          user,
//          success: true,
//        });
//    } catch (error) {
//      console.error(error);
//      res.status(500).json({
//        message: "Server Error login failed",
//        success: false,
//      });
//    }
//  }
// };

// export const logout = async (req, res) => {
//   try {
//     return res.status(200).cookie("token", "", { maxAge: 0 }).json({
//       message: "Logged out successfully.",
//       success: true,
//     });
//   } catch (error) {
//     console.log(error);
//   }
// };

// export const updateProfile = async (req, res) => {
//   try {
//     console.log("Uploaded file:", req.file);
//     console.log("Request body:", req.body);

//     const { fullname, email, phoneNumber, bio, skills } = req.body;
//     const file = req.file;

//     // Check if file is uploaded

//     //cloudinary upload
//     const fileUri = getDataUri(file);
//     const cloudResponse = await cloudinary.uploader.upload(fileUri.content);

//     // Initialize userId at the beginning
//     const userId = req.id; // middleware authentication

//     // Check if userId is valid
//     let user = await User.findById(userId);
//     if (!user) {
//       return res.status(404).json({
//         message: "User not found",
//         success: false,
//       });
//     }

//     // Process skills if provided
//     let skillsArray;
//     if (skills) {
//       skillsArray = skills.split(",");
//     }

//     // Update user profile
//     if (fullname) {
//       user.fullname = fullname;
//     }
//     if (email) {
//       user.email = email;
//     }
//     if (phoneNumber) {
//       user.phoneNumber = phoneNumber;
//     }
//     if (bio) {

```

```

//      user.profile.bio = bio;
//    }
//    if (skills) {
//      user.profile.skills = skillsArray;
//    }
//    //resume
//    if (cloudResponse) {
//      user.profile.resume = cloudResponse.secure_url;
//      user.profile.resumeOriginalName = file.originalname;
//    }

//    // Save updated user
//    await user.save();

//    user = {
//      _id: user._id,
//      fullname: user.fullname,
//      email: user.email,
//      phoneNumber: user.phoneNumber,
//      role: user.role,
//      profile: user.profile,
//    };

//    return res.status(200).json({
//      message: "Profile updated successfully",
//      user,
//      success: true,
//    });
//  } catch (error) {
//    console.error(error);
//    res.status(500).json({
//      message: "Server Error updating profile",
//      success: false,
//    });
//  }
// }
// };

```

Backend\generate_pdf.js

```

import fs from 'fs';
import path from 'path';
import PDFDocument from 'pdfkit';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const rootDir = path.join(__dirname, '..');

const doc = new PDFDocument({ margin: 40, size: 'A4', autoFirstPage: true });
const outputPath = path.join(rootDir, 'Internship_Portal_Source_Code.pdf');
doc.pipe(fs.createWriteStream(outputPath));

const directories = [
  'Backend/controllers',
  'Backend/models',
  'Backend/routes',
  'Backend/utils',
  'Backend/middleware',
  'Backend/index.js',
  'Frontend/src',
  'Frontend/index.html'
];

doc.fontSize(24).text('Internship Portal - Source Code', { align: 'center' });
doc.moveDown();
doc.fontSize(12).text(`Generated on: ${new Date().toLocaleString()}`, { align: 'center' });
doc.moveDown(3);
doc.fontSize(14).text('This document contains the complete frontend and backend source code for the Internship Portal web application, organized by file.', { align: 'center' });

function walkSync(currentDirPath, callback) {
  if (!fs.existsSync(currentDirPath)) return;

  const stat = fs.statSync(currentDirPath);
  if (stat.isFile()) {
    callback(currentDirPath);
    return;
  }

  fs.readdirSync(currentDirPath).forEach(function (name) {
    var filePath = path.join(currentDirPath, name);
    var stat = fs.statSync(filePath);
    if (stat.isFile()) {
      if (filePath.match(/\.js|jsx|css|html$/)) && !filePath.includes('node_modules')) {
        callback(filePath);
      }
    } else if (stat.isDirectory()) {
      if (!filePath.includes('node_modules') && !filePath.includes('dist') && !filePath.includes('components/ui')) {
        // Skipping components/ui which are usually shadcn generated components, keeping core logic
        walkSync(filePath, callback);
      }
    }
  });
}

directories.forEach(dir => {
  const fullPath = path.join(rootDir, dir);
  walkSync(fullPath, (filePath) => {
    let content = fs.readFileSync(filePath, 'utf8');

    // Clean up some problematic characters that might break pdfkit Courier font
    content = content.replace(/\\r\\n/g, '\\n');

    doc.addPage();
    doc.fontSize(16).fillColor('#E50914').text(`File: ${path.relative(rootDir, filePath).replace(/\\/g, '/')}`, { underline: true });
  });
  doc.moveDown();

  doc.fontSize(9).fillColor('#000000').font('Courier').text(content, {
    lineBreak: true,
    columns: 1
  });
});

doc.end();
console.log(`☑ PDF generated successfully at: ${outputPath}`);

```

Backend\\index.js

```

import express from "express";
import cookieParser from "cookie-parser";
import cors from "cors";
import dotenv from "dotenv";
import connectDB from "./utils/db.js";
import userRoute from "./routes/user.route.js";
import companyRoute from "./routes/company.route.js";
import jobRoute from "./routes/job.route.js";
import applicationRoute from "./routes/application.route.js";

dotenv.config({});
const app = express();

//middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cookieParser());

const corsOptions = {
  origin: ["http://localhost:5173", "http://127.0.0.1:5173"],
  credentials: true,
};

app.use(cors(corsOptions));

const PORT = process.env.PORT || 5001;

//api's

app.use("/api/user", userRoute);
app.use("/api/company", companyRoute);
app.use("/api/job", jobRoute);
app.use("/api/application", applicationRoute);

app.listen(PORT, () => {
  connectDB();
  console.log(`Server is running on port ${PORT}`);
});

```

Backend\middleware\isAuthenticated.js

```

import jwt from "jsonwebtoken";

const authenticateToken = (req, res, next) => {
  try {
    const token = req.cookies.token;
    if (!token) {
      return res
        .status(401)
        .json({ message: "No token provided", success: false });
    }
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    if (!decoded) {
      return (
        res.status(401).json({ message: "Invalid token" }),
        (success = false)
      );
    }
    req.id = decoded.userId;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid token" });
  }
};

export default authenticateToken;

```

Backend\middleware\multer.js

```

import multer from "multer";

const storage = multer.memoryStorage();
export const singleUpload = multer({ storage }).single("file");

```

Backend\models\application.model.js

```

import mongoose from "mongoose";
const applicationSchema = new mongoose.Schema(
  {
    job: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Job",
      required: true,
    },
    applicant: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    status: {
      type: String,
      enum: ["pending", "accepted", "rejected"],
      default: "pending",
    },
  },
  {
    timestamps: true,
  }
);

export const Application = mongoose.model("Application", applicationSchema);

```

Backend\models\company.model.js

```

import mongoose from "mongoose";

const companySchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    unique: true
  },
  description: {
    type: String,
  },
  website: {
    type: String
  },
  location: {
    type: String
  },
  logo: {
    type: String // URL to company logo
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, {timestamps: true})

export const Company = mongoose.model("Company", companySchema);

```

Backend\models\job.model.js


```
import mongoose from "mongoose";

const jobSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
    },
    description: {
      type: String,
      required: true,
    },
    requirements: [
      {
        type: String,
      },
    ],
    salary: {
      type: String,
      required: true,
    },
    experienceLevel: {
      type: Number,
      required: true,
    },
    location: {
      type: String,
      required: true,
    },
    jobType: {
      type: String,
      required: true,
    },
    position: {
      type: Number,
      required: true,
    },
    company: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Company",
      required: true,
    },
    created_by: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    applications: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Application",
      },
    ],
  },
  { timestamps: true }
);

export const Job = mongoose.model("Job", jobSchema);
```

Backend\models\user.model.js

```

import mongoose from "mongoose";
const userSchema = new mongoose.Schema(
  {
    fullname: {
      type: String,
      required: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
    },
    phoneNumber: {
      type: String,
      required: true,
      unique: true,
    },
    password: {
      type: String,
      required: true,
    },
    pancard: {
      type: String,
      required: true,
      unique: true,
    },
    adharcard: {
      type: String,
      required: true,
      unique: true,
    },
    role: {
      type: String,
      enum: ["Student", "Recruiter"],
      default: "Student",
      required: true,
    },
    profile: {
      bio: {
        type: String,
      },
      skills: [{ type: String }],
      resume: {
        type: String, // URL to resume file
      },
      resumeOriginalname: {
        type: String, // Original name of resume file
      },
      resumeAnalysis: {
        score: { type: Number },
        strengths: [{ type: String }],
        improvements: [{ type: String }]
      },
      company: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "Company",
      },
      profilePhoto: {
        type: String, // URL to profile photo file
        default: "",
      },
    },
  },
  { timestamps: true }
);

export const User = mongoose.model("User", userSchema);

```

Backend\routes\application.route.js

```

import express from "express";

import authenticateToken from "../middleware/isAuthenticated.js";
import { applyJob, getApplicants, getAppliedJobs, updateStatus } from "../controllers/application.controller.js";

const router = express.Router();

router.route("/apply/:id").get(authenticateToken, applyJob);
router.route("/get").get(authenticateToken, getAppliedJobs);
router.route("/:id/applicants").get(authenticateToken, getApplicants);
router.route("/status/:id/update").post(authenticateToken, updateStatus);

export default router;

```

Backend\routes\company.route.js

```
import express from "express";

import authenticateToken from "../middleware/isAuthenticated.js";
import {
  getAllCompanies,
  getCompanyById,
  registerCompany,
  updateCompany,
} from "../controllers/company.controller.js";
import { singleUpload } from "../middleware/multer.js";

const router = express.Router();

router.route("/register").post(authenticateToken, registerCompany);
router.route("/get").get(authenticateToken, getAllCompanies);
router.route("/get/:id").get(authenticateToken, getCompanyById);
router.route("/update/:id").put(authenticateToken, singleUpload, updateCompany);

export default router;
```

Backend\routes\job.route.js

```
import express from "express";

import authenticateToken from "../middleware/isAuthenticated.js";
import {
  getAdminJobs,
  getAllJobs,
  getJobById,
  postJob,
} from "../controllers/job.controller.js";

const router = express.Router();

router.route("/post").post(authenticateToken, postJob);
router.route("/get").get(getAllJobs);
router.route("/getadminjobs").get(authenticateToken, getAdminJobs);
router.route("/get/:id").get(getJobById);
export default router;
```

Backend\routes\user.route.js

```
import express from "express";
import {
  login,
  logout,
  register,
  updateProfile,
} from "../controllers/user.controller.js";
import authenticateToken from "../middleware/isAuthenticated.js";
import { singleUpload } from "../middleware/multer.js";

const router = express.Router();

router.route("/register").post(singleUpload, register);
router.route("/login").post(login);
router.route("/logout").post(logout);
router
  .route("/profile/update")
  .post(authenticateToken, singleUpload, updateProfile);

export default router;
```

Backend\seed.js

```
import mongoose from "mongoose";
import dotenv from "dotenv";
import bcrypt from "bcryptjs";
import { User } from "../models/user.model.js";
import { Company } from "../models/company.model.js";
import { Job } from "../models/job.model.js";

dotenv.config();

const seedData = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected for seeding...");
```

```

// Clear existing data
await Job.deleteMany({});
await Company.deleteMany({});
const existingRecruiter = await User.findOne({ email: "recruiter@internshipportal.com" });
if (existingRecruiter) {
  await User.deleteOne({ email: "recruiter@internshipportal.com" });
}

// Create a recruiter user
const hashedPassword = await bcrypt.hash("password123", 10);
const recruiter = await User.create({
  fullname: "Portal Recruiter",
  email: "recruiter@internshipportal.com",
  phoneNumber: "9999999999",
  password: hashedPassword,
  pancard: "SEED12345A",
  adharcard: "999999999999",
  role: "Recruiter",
  profile: {
    bio: "Official recruiter account for seeding internships",
    profilePhoto: "",
  },
});

console.log("✅ Recruiter created");

// Create companies
const companies = await Company.insertMany([
  {
    name: "Google",
    description: "A global technology leader specializing in search, cloud, and AI.",
    website: "https://careers.google.com",
    location: "Bangalore",
    logo: "https://logo.clearbit.com/google.com",
    userId: recruiter._id,
  },
  {
    name: "Microsoft",
    description: "World's leading software company building cloud and productivity tools.",
    website: "https://careers.microsoft.com",
    location: "Hyderabad",
    logo: "https://logo.clearbit.com/microsoft.com",
    userId: recruiter._id,
  },
  {
    name: "Amazon",
    description: "Global e-commerce and cloud computing giant.",
    website: "https://amazon.jobs",
    location: "Bangalore",
    logo: "https://logo.clearbit.com/amazon.com",
    userId: recruiter._id,
  },
  {
    name: "Flipkart",
    description: "India's leading e-commerce marketplace.",
    website: "https://www.flipkartcareers.com",
    location: "Bangalore",
    logo: "https://logo.clearbit.com/flipkart.com",
    userId: recruiter._id,
  },
  {
    name: "Razorpay",
    description: "India's leading fintech company for payment solutions.",
    website: "https://razorpay.com/careers",
    location: "Bangalore",
    logo: "https://logo.clearbit.com/razorpay.com",
    userId: recruiter._id,
  },
  {
    name: "Zomato",
    description: "India's top food delivery and restaurant discovery platform.",
    website: "https://www.zomato.com/careers",
    location: "Gurugram",
    logo: "https://logo.clearbit.com/zomato.com",
    userId: recruiter._id,
  },
]);

console.log(`✅ ${companies.length} companies created`);

// Create internships/jobs
const jobs = await Job.insertMany([
  {
    title: "Frontend Developer Intern",
    description: "Build beautiful user interfaces using React.js and modern CSS frameworks. Work with the design team to implement responsive, pixel-perfect UIs.",
  },
]);

```

```

    requirements: ["React.js", "HTML/CSS", "JavaScript", "Git"],
    salary: "6",
    experienceLevel: 0,
    location: "Bangalore",
    jobType: "Full-time",
    position: 5,
    company: companies[0]._id, // Google
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "Backend Developer Intern",
    description: "Design and develop RESTful APIs and microservices. Work with Node.js, Express, and MongoDB to build scalable
backend systems.",
    requirements: ["Node.js", "Express", "MongoDB", "REST APIs"],
    salary: "8",
    experienceLevel: 0,
    location: "Hyderabad",
    jobType: "Full-time",
    position: 3,
    company: companies[1]._id, // Microsoft
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "Full Stack Developer Intern",
    description: "End-to-end development of web applications. Build features from database design to frontend implementation
using the MERN stack.",
    requirements: ["React", "Node.js", "MongoDB", "Express", "TypeScript"],
    salary: "10",
    experienceLevel: 1,
    location: "Bangalore",
    jobType: "Full-time",
    position: 2,
    company: companies[2]._id, // Amazon
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "Data Science Intern",
    description: "Analyze large datasets to extract actionable insights. Build machine learning models and create data
visualizations for business decisions.",
    requirements: ["Python", "Pandas", "Machine Learning", "SQL"],
    salary: "7",
    experienceLevel: 0,
    location: "Bangalore",
    jobType: "Full-time",
    position: 4,
    company: companies[3]._id, // Flipkart
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "DevOps Engineer Intern",
    description: "Manage CI/CD pipelines, containerize applications with Docker, and deploy to cloud platforms. Automate
infrastructure provisioning.",
    requirements: ["Docker", "AWS", "Linux", "CI/CD", "Kubernetes"],
    salary: "9",
    experienceLevel: 1,
    location: "Pune",
    jobType: "Full-time",
    position: 2,
    company: companies[0]._id, // Google
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "UI/UX Design Intern",
    description: "Create intuitive user experiences and stunning visual designs. Conduct user research, build wireframes, and
prototype interactions in Figma.",
    requirements: ["Figma", "Adobe XD", "User Research", "Prototyping"],
    salary: "5",
    experienceLevel: 0,
    location: "Mumbai",
    jobType: "Part-time",
    position: 3,
    company: companies[4]._id, // Razorpay
    created_by: recruiter._id,
    applications: [],
  },
  {
    title: "Machine Learning Intern",
    description: "Develop and train ML models for recommendation systems and NLP. Work with TensorFlow and PyTorch on real-world
production datasets.",
    requirements: ["Python", "TensorFlow", "PyTorch", "NLP", "Statistics"],
    salary: "12",

```

```

        experienceLevel: 1,
        location: "Bangalore",
        jobType: "Full-time",
        position: 2,
        company: companies[1]._id, // Microsoft
        created_by: recruiter._id,
        applications: [],
    },
    {
        title: "Mobile App Developer Intern",
        description: "Build cross-platform mobile applications using React Native. Implement features, optimize performance, and
publish to app stores.",
        requirements: ["React Native", "JavaScript", "iOS", "Android"],
        salary: "7",
        experienceLevel: 0,
        location: "Gurugram",
        jobType: "Full-time",
        position: 3,
        company: companies[5]._id, // Zomato
        created_by: recruiter._id,
        applications: [],
    },
    {
        title: "Cybersecurity Intern",
        description: "Perform vulnerability assessments, security audits, and penetration testing. Help secure cloud infrastructure
and web applications.",
        requirements: ["Network Security", "Linux", "Python", "OWASP"],
        salary: "8",
        experienceLevel: 0,
        location: "Chennai",
        jobType: "Full-time",
        position: 2,
        company: companies[2]._id, // Amazon
        created_by: recruiter._id,
        applications: [],
    },
    {
        title: "Product Management Intern",
        description: "Define product roadmaps, gather user requirements, and work with engineering teams to ship features. Analyze
metrics and drive product growth.",
        requirements: ["Product Strategy", "Analytics", "Communication", "Agile"],
        salary: "6",
        experienceLevel: 0,
        location: "Remote",
        jobType: "Part-time",
        position: 2,
        company: companies[4]._id, // Razorpay
        created_by: recruiter._id,
        applications: [],
    },
    },
    });

    console.log(`✅ ${jobs.length} internships created`);
    console.log("\n🎉 Seeding complete! Your portal now has sample data.");
    console.log("    You can now refresh http://localhost:5173 to see the internships.\n");

    process.exit(0);
} catch (error) {
    console.error("❌ Seeding failed:", error.message);
    process.exit(1);
}
};

seedData();

```

Backend\utils\cloud.js

```

import { v2 as cloudinary } from "cloudinary";

import dotenv from "dotenv";
dotenv.config();

cloudinary.config({
    cloud_name: process.env.CLOUD_NAME,
    api_key: process.env.CLOUD_API,
    api_secret: process.env.API_SECRET,
});

export default cloudinary;

```

Backend\utils\datauri.js

```
import DataUriParser from "datauri/parser.js"

import path from "path";

const getDataUri = (file) => {
  const parser = new DataUriParser();
  const extName = path.extname(file.originalname).toString();
  return parser.format(extName, file.buffer);
}

export default getDataUri;
```

Backend\utils\db.js

```
import mongoose from "mongoose";

// Connect to MongoDB
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected...");
  } catch (error) {
    console.error("Error connecting to MongoDB:", error.message);
    process.exit(1);
  }
};

export default connectDB;
```